Steganography Toolkit for Secure Image and Audio Messaging

Project Phase-II Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

Information Technology

By

Mr. MOHAMMED MUBASHEERUDDIN NASER (160922737033)

Mr. MOHAMMED ANAS ABDUL WARIS (160922737029)

Mr. MOHAMMED FASI (160922737050)

Under the Guidance of

Mr. Mohammed Mateen Ahmmed

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University)

Himayathsagar, Hyderabad,500091

A.Y. 2025-2026

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-II report entitled **Steganography Toolkit For Secure Image And Audio Messaging** being submitted by **Mr. Mohammed Mubasheeruddin Naser (160922737033), Mr. Mohammed Anas Abdul Waris (160922737029), Mr. Mohammed Fasi (160922737050)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of bonafide work carried out by him.

Date:

Internal Project Guide
Mr. Mohammed Mateen Ahmmed **Dr. K. Nagi Reddy**
Assistant Professor **H.O.D, I.T**

External Examiner

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

DECLARATION BY THE CANDIDATES

We, **Mr. Mohammed Mubasheeruddin Naser (160922737033), Mr. Mohammed Anas Abdul Waris (160922737029) and Mr. Mohammed Fasi (160922737050)**, hereby declare that the Project Phase- II report entitled **STEGANOGRAPHY TOOLKIT FOR SECURE IMAGE AND AUDIO MESSAGING** is carried out under the guidance of **Mr. Mohammed Mateen Ahmmed** Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology**, **Hyderabad** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Mohammed Mubasheeruddin Naser (160922737033)
Mohammed Anas Abdul Waris (160922737029)
Mohammad Fasi (16092273050)

ACKNOWLEDGMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding a number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express my deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Mr. Mohammed Mateen Ahmmed, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for his guidance and help. His insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Mohammed Mubasheeruddin Naser (160922737033)
Mohammed Anas Abdul Waris (160922737029)
Mohammad Fasi (160922737050)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad 91

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adopt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad 91

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

Course Outcomes: PP.CO- Project Phase- II
After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program related to the real-world problems.	**Apply**
PP.CO2	Organize and examine different solutions based on economic, technical and functional feasibilities.	**Analyse**
PP.CO3	Provide support and interpret the critics for implementing the better solutions to formulate the robust project.	**Evaluate**
PP.CO4	Design and develop a model using technical and functional provisions to make the formulated solutions.	**Create**
PP.CO5	Compose the project components and apply documentation standards to prepare the project report and present the project work.	**Create**

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.CO1	3	2	-	2	1	-	-	-	-	-	2	3	2
PP.CO2	2	3	2	3	-	-	-	-	-	1	2	2	3
PP.CO3	3	2	3	3	2	2	-	-	-	-	3	3	3
PP.CO4	3	2	3	1	3	2	3	3	3	2	3	3	3
PP.CO5	2	-	3	-	2	1	3	3	2	2	3	3	3
Average	2.6	2.25	2.75	2.25	2.66	1.66	3	3	2.5	1.6	2.6	2.8	2.8

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION		10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

The Steganography Toolkit for Secure Image and Audio Messaging is designed to provide a secure, efficient, and user-friendly platform for hiding confidential messages inside digital image and audio files, enabling covert communication with enhanced privacy and protection. The system integrates Advanced Encryption Standard (AES) encryption, media preprocessing, and steganographic embedding techniques to create a robust and multi-layered security mechanism for secret data transmission. In the embedding phase, users can upload a cover image or audio file, enter a confidential message, encrypt it using AES, and embed the encrypted data into the selected media through the Steganography Embedding Engine. The generated output is a stego file that appears visually or audibly almost identical to the original cover media, thereby preserving secrecy and minimizing suspicion. To ensure consistency and reliability, the toolkit also performs preprocessing operations such as file normalization, format adjustment, and media handling optimization before embedding. In the extraction phase, the reverse workflow is applied, where the stego image or audio file is processed to identify and retrieve the hidden encrypted bits, which are then decrypted using the correct key to recover the original message accurately. The system further enhances security by incorporating a server-side Key Management System (KMS) that securely stores, retrieves, and manages cryptographic keys, reducing the risk of insecure key handling and unauthorized access. The overall architecture provides end-to-end security through encrypted key storage, secure message transformation, hidden data embedding, and tamper-resistant extraction pipelines, ensuring both confidentiality and undetectability. By combining the strengths of cryptography and steganography, the proposed toolkit offers a practical solution for modern secure communication needs and is highly suitable for applications in intelligence operations, digital forensics, military and defense communication, personal privacy protection, secure corporate communication, and other domains where hidden and protected information exchange is essential.

CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Vision and Mission of the	iv
Institute Vision and Mission of the	v
Department Program Educational	vi
Objectives Program Outcomes	vii
Program Specific Outcomes	viii
Course Outcomes	ix
Course Articulation Matrix	x
Abstract	xi
Contents	xii
List of Figures	xiv
List of Tables	xv

CHAPTER-1: INTRODUCTION	**1**
1.1 Filter functionality	2
1.2 Background Functionality	3
CHAPTER-2: LITERATURE SURVEY	**6**
CHAPTER-3: SYSTEM ANALYSIS AND REQUIREMENTS	**9**
3.1 Existing System Analysis	9
3.2 Drawbacks of Existing System	10
3.3 System Requirements	11
CHAPTER-4: SYSTEM STUDY AND DESIGN	**15**
4.1 System Architecture	15
4.2 Feasibility System Study	18
4.3 Feasibility System Design	20
CHAPTER-5: IMPLEMENTATION	**23**
5.1 Modules	23
5.2 Python	24
5.3 Flask	24

5.4 Sample Code 26
CHAPTER-6: INPUT AND OUTPUT DESIGN 45
6.1 System Input Design 45
6.2 System Output Design 47
CHAPTER-7: SOFTWARE ENVIRONMENT 51
7.1 Software Environment 51
7.2 Software Installation 54
CHAPTER-8: SCREENSHOTS 58
8.1 Login Page 58
8.2 Dashboard 58
8.3 Activity Graph 59
8.4 Encoding Process 59
8.5 Checksum Verification 60
8.6 Risk Analysis 60
8.7 History Page 61
8.8 Admin Page 61
CHAPTER-9: SYSTEM TESTING AND RESULTS DISCUSSION 62
9.1 System Testing 62
9.2 System Results 65
9.3 System Discussion 66
CHAPTER-10: CONCLUSION & FUTURE ENHANCEMENT 69
10.1 Future Scope 69
10.2 Future Enhancements 70
CHAPTER-11: REFERENCES 73

List of Figures

SI.no	Fig.no	Name of the Figure	Page No
1	4.1.1	System Architecture	15
2	4.1.2	Work Flow Diagram	18
3	5.3.1	Work Flow of Flask Architecture	25
4	5.3.2	Flask Admin Model	25
5	8.1	Login Page	58
6	8.2	Dashboard	58
7	8.3	Activity Graph	59
8	8.4	Encoding Process	59
9	8.5	Checksum Verification	60
10	8.6	Risk Analysis	60
11	8.7	History Page	61
12	8.8	Admin Page	61

List of Tables

SI.no	Table.no	Name of the Table	Page No
1	9.3	Performance Evaluation Metrics of the Proposed system	68

CHAPTER 1
INTRODUCTION

In the modern digital environment, secure communication has become a primary concern due to the increasing threats of data interception, surveillance, cyberattacks, and unauthorized access. As digital communication continues to grow across personal, academic, business, and governmental platforms, the need to protect sensitive information has become more important than ever. Traditional encryption techniques are highly effective in securing the content of a message by converting it into an unreadable format, but they do not hide the existence of the communication itself. In many situations, the presence of encrypted data may attract suspicion and increase the risk of interception or analysis by attackers. To overcome this limitation, steganography provides an additional level of protection by concealing secret information within ordinary digital media such as images and audio files. Unlike encryption, which only protects the message content, steganography hides the message in a way that makes it invisible to unintended users. This makes it a highly valuable technique for covert communication, where both secrecy and invisibility are required. By combining the strengths of cryptography and steganography, a more secure and intelligent communication system can be developed that not only protects the information but also hides its very presence from external observers.

The Steganography Toolkit for Secure Image and Audio Messaging is developed as a secure communication system that enables users to hide encrypted messages inside image and audio files for safe and covert transmission. The toolkit uses Advanced Encryption Standard (AES) to encrypt the secret message before embedding it into the selected media, thereby ensuring that the hidden data remains unreadable even if extracted by an unauthorized person. The system follows a clear workflow in which the user uploads a cover image or audio file, enters a secret message, encrypts it, and embeds it using a Steganography Engine to generate a stego file. During extraction, the reverse process is performed: the stego media is uploaded, the hidden bits are retrieved, decrypted, and presented securely to the authorized user. The system may also include a Key Management Service (KMS) to securely store and manage encryption keys, further enhancing overall security and usability. By supporting both image and audio steganography, the toolkit offers flexibility, improved practicality, and broader application potential. This project demonstrates how the integration of data encryption, media preprocessing, and information hiding techniques can create a powerful solution for secure communication in sensitive areas such as confidential messaging, digital forensics, military communication, secure file sharing, and personal privacy protection.

Overall, the proposed Steganography Toolkit provides a comprehensive solution for secure multimedia communication by combining encryption, data hiding, and integrity verification within a user-friendly platform.

1.1 Filter Functionality

The system uses multiple filtering functions throughout the workflow to ensure only valid, secure, and correctly formatted data passes through each component. These filters prevent errors, malicious activity, improper encoding, and data corruption.

1. Input Media Filter

- Accepts only supported image/audio formats (PNG, JPG, WAV, MP3).
- Rejects corrupted or unsupported files.
- Prevents malformed media from entering the embedding pipeline.

2. Message Input Filter

- Sanitizes user-entered messages.
- Blocks harmful characters or scripts (e.g., HTML, SQL, JavaScript).
- Checks message length against payload capacity of the media.

3. Preprocessing Filter

- Standardizes resolution, bit depth, and sampling rate.
- Ensures the media is suitable for embedding.
- Removes inconsistencies that can break steganographic algorithms.

4. Encryption Filter

- Ensures AES encryption receives valid input.
- Rejects empty messages or corrupted inputs.
- Verifies key retrieval from KMS before encryption.

5. Embedding Filter

- Ensures encrypted data fits into the selected media.
- Prevents embedding that would distort audio/image quality.
- Validates embedding method (LSB or custom algorithm).

6. Stego Media Verification Filter

- Checks stego file for structural integrity before output

- Ensures no corruption occurred during embedding.

7. Extraction Filter

- Validates uploaded stego media.
- Ensures it contains properly embedded data.
- Filters out invalid or tampered stego files.

8. Decryption Filter

- Confirms extracted data is valid encrypted content.
- Rejects modified or incomplete encrypted payload.
- Prevents decryption with invalid or mismatched keys.

These filters ensure smooth functioning and act as security checkpoints throughout the system.

1.2 Background Functionality

The system architecture is divided into two major parts:

A. Client Side (Primary Operations)

This is where all visible user interaction and hidden internal processing occur.

1. User Interface

- The starting point where the user uploads cover images/audio files.
- Allows the user to enter the secret message or upload stego media for extraction.

2. Image/Audio Import Module

- Reads the uploaded media file.
- Verifies the file type, quality, and compatibility.
- Sends the media to the preprocessing module.

3. Preprocessing Module

- Standardizes media characteristics such as:
  - Image resolution
  - Bit depth

- Audio sampling rat

- Ensures the file is optimal for embedding or extraction.
- Removes metadata or structures that could interfere with embedding.

4. AES Encryption Module

- Takes the user's text message as input.
- Retrieves encryption keys from the **KMS** (if configured).
- Encrypts message using AES-256.
- Outputs an encrypted message ready for embedding.

5. Steganography Embedding Engine

- Core component for data hiding.
- Embeds the encrypted message into:
- Image pixels (LSB image steganography), or
- Audio samples (LSB or spread spectrum).
- Ensures embedding does not noticeably change the visual or audio quality.
- Outputs a stego image or stego audio file

6. Stego Media Output

- Provides downloadable/exportable stego files.
- Stego media retains the appearance and sound of the original file.

7. Decode + Decrypt Module (Extraction Side)

- Accepts uploaded stego media.
- Extracts the hidden embedded bits.
- Sends extracted encrypted data to AES module for decryption.
- Retrieves the secret message and displays it securely.

This ensures complete end-to-end functionality of message hiding and retrieval.

B. Server Side (KMS + Encrypted Key Vault)

Although optional, the server side significantly increases cryptographic security.

1. Key Management System (KMS)

- Handles key generation, retrieval, and storage.
- Communicates securely via TLS.
- Ensures only authenticated client-side modules can retrieve keys.

2. Encrypted Key Vault

- Stores AES keys in secure, encrypted form.
- Ensures keys are never exposed in plaintext.
- Provides centralized control of cryptographic lifecycle.

The KMS integration enhances security, especially for multi-user or enterprise systems.

Overall Process:

The entire architecture ensures:

- Confidentiality through AES encryption
- Undetectability through steganography
- Integrity through preprocessing and controlled embedding
- Secure key handling through KMS
- Accurate extraction and decryption.

CHAPTER 2
LITERATURE SURVEY

1. Title: "A Two-Phase Embedding Approach for Secure Distributed Steganography"

Authors: Kamil Woźniak, Marek R. Ogiela, Lidia Ogiela

Year: 2025

Summary:

This paper presents an advanced distributed steganography technique that improves secure hidden communication by dividing a secret message into multiple smaller parts and embedding them across different digital media files instead of hiding the entire message inside a single cover file. The authors introduce a two-phase embedding process, where the first phase focuses on preparing and segmenting the secret data, while the second phase handles the actual embedding of these segments into selected carriers. This approach significantly improves security, because even if one stego file is intercepted or detected, the attacker cannot retrieve the full hidden message. The paper also discusses the importance of message fragmentation, distribution strategy, and reconstruction logic, which together make the system more robust and resistant to steganalysis. By spreading the hidden data across multiple files, the method reduces suspicious modifications in any one file and improves overall undetectability. This research is highly relevant to the proposed project because it supports the concept of secure covert communication and demonstrates how steganographic systems can be made stronger through structured embedding strategies and improved secrecy.

2. Title: "Deep Learning Based Secure Image Steganography Using U-Net Architecture"

Authors: Peng Ji, Youlue Zhang, Zhongyou Lv

Year: 2024

Summary:

This paper introduces a modern image steganography system based on deep learning, specifically using a U-Net neural network architecture, to improve the embedding and extraction of secret messages in digital images. Unlike traditional steganographic methods such as Least Significant Bit (LSB) insertion, which follow fixed rules for data hiding, the proposed deep learning model automatically learns the most suitable image regions and patterns for secure embedding. The system is designed to optimize major steganographic requirements such as high embedding capacity, low visual distortion, accurate extraction, and resistance to steganalysis. The paper explains that the neural network analyzes image textures, edges, and pixel distributions to determine where hidden information can be inserted with minimal visual impact. Experimental results show that this method can hide more data while preserving the natural appearance of the image and improving recovery accuracy.

This paper is highly useful for the proposed project because it reflects the latest trend in image steganography and can be used to explain future enhancements in which the image hiding process can be made more intelligent, adaptive, and secure using AI-based techniques.

3. Title: "Robust Steganography in Practical Communication: A Comparative Study"

Authors: Tong Qiao, Shengwang Xu, Shuai Wang, Xiaoshuai Wu, Bo Liu, Ning Zheng, Ming Xu,

Year: 2023

Summary:

This paper provides a detailed comparative study of different robust steganography techniques and evaluates how they perform in practical communication environments such as Facebook, Twitter, and WeChat, where images are often compressed or modified after upload. The authors explain that many steganographic systems perform well in laboratory conditions but fail in real-world situations because hidden messages can be damaged during compression, transmission, or format changes. The paper compares various methods using important parameters such as capacity, imperceptibility, undetectability, and robustness, and concludes that an effective steganographic system must preserve the hidden data even after common platform-level processing. This paper is very useful for your project because it supports the importance of media preprocessing, quality preservation, and reliable extraction, especially if your toolkit is designed for practical secure communication rather than only theoretical implementation.

4. Title: "Robust Image Steganography Against Lossy JPEG Compression Based on Embedding Domain Selection and Adaptive Error Correction"

Authors: Xiaolong Duan, Bin Li, Zhaoxia Yin, Xinpeng Zhang, Bin Luo

Year: 2023

Summary:

This paper focuses on improving image steganography robustness in situations where images are compressed using JPEG, which is a common challenge in practical communication systems and social media sharing. The authors propose a method based on embedding domain selection and adaptive error correction, where the system intelligently selects the most stable regions of the image for embedding and applies error correction techniques to protect the hidden message. The study aims to improve anti- steganalysis performance, hidden data survival, and message recovery accuracy after lossy compression. The results show that the proposed method performs better than many traditional techniques when images undergo recompression or online sharing. This research also emphasizes the importance of maintaining a balance between security, embedding capacity, and image quality in steganographic systems. It shows that selecting appropriate embedding regions can significantly reduce the chances of hidden data being damaged during file compression or transmission. The paper further demonstrates that adaptive error correction plays a major role in ensuring accurate recovery of the secret message even after image modifications. These

findings are especially useful for designing real-world secure communication tools where media files may be uploaded, shared, or processed multiple times. Overall, the study provides valuable insights for building a more robust and reliable image steganography module in the proposed toolkit.

5. Title: "A Novel Steganography Approach for Audio Files"

Authors: Sazeen T. Abdulrazzaq, Mohammed M. Siddeq, Marcos A. Rodrigues

Year: 2020

Summary:

This paper presents a secure audio steganography technique for hiding secret information inside WAV audio files using a Least Significant Bit (LSB) based method. The authors aim to increase the carrier capacity of audio files while preserving original sound quality and maintaining high imperceptibility so that the hidden data remains inaudible to human listeners. The method also involves compressing image data using a GMPR technique based on DCT before embedding it into audio, which improves storage efficiency and hiding performance. The paper emphasizes important performance factors such as embedding capacity, signal quality, and successful extraction, making it highly relevant to systems that support audio-based covert communication. This research is directly useful for your project because your toolkit includes audio secure messaging, and this paper provides a strong technical basis for hiding encrypted secret messages inside sound files. The study also shows that audio files can act as effective cover media because they contain a large number of samples for data embedding. It highlights that careful modification of audio bits can hide information without causing noticeable distortion in playback. The paper further explains the importance of balancing security, embedding strength, and audio quality for reliable hidden communication. It is especially useful for applications where secret messages need to be transmitted in an undetectable manner. Overall, this research supports the development of a strong and practical audio steganography module in the proposed toolkit.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System Analysis

Traditional communication systems rely heavily on encryption alone to secure messages. While encryption protects content, it does not hide the fact that a secret message exists. Encrypted messages can attract attention, especially in environments with surveillance, monitoring, or data inspection. Existing steganography systems also suffer from several limitations:

A. Plain Steganography (Without Encryption)

Many tools hide plaintext directly into images or audio:

- Hidden data becomes vulnerable if extracted.
- Anyone with extraction knowledge can reveal the message.
- No confidentiality if stego media is intercepted.

B. Basic LSB Tools

There are numerous simple LSB-based tools:

- Offer no preprocessing or format normalization.
- Fail on compressed or transformed media.
- Unable to detect tampering or corruption.

C. No Key Management or Secure Key Storage

Most existing systems:

- Store keys locally in plaintext.
- Lack centralized or secure key management.
- Have no authentication for key retrieval.

D. Limited Support for Audio Steganography

Many tools support only image steganography and lack:

- Audio cover support

- Audio normalization
- Audio quality preservation

E. No End-to-End Workflow

Existing tools do not provide:

- A complete embed → share → extract → decrypt pipeline.
- Integration of encryption + steganography + key management.

3.2 Drawbacks of Existing System:

1. **No encryption before embedding**

If hidden data is extracted, the message becomes readable.

2. **No preprocessing**

Media distortions result in corruption of hidden data.

3. **No secure key lifecycle**

Keys are stored insecurely or manually handled.

4. **Low robustness**

Embedded data is easily corrupted during file compression or editing.

5. **Single-direction workflow**

Tools often embed messages but lack reliable extraction tools.

6. **No quality preservation mechanisms**

Embedding may distort images or audio noticeably.

7. **Limited media compatibility**

Certain formats fail silently or become corrupted.

Your proposed toolkit resolves these drawbacks by integrating AES encryption, preprocessing, secure key management (KMS), bidirectional workflow, and high-quality steganography.

3.3 System Requirements

System requirements outline functional, non-functional, hardware, and software needs for implementing the toolkit as represented in your architecture.

A. Functional Requirements

These define what the system must do.

1. User Interface Requirements

- Allow user to upload cover image/audio.
- Allow user to enter secret message.
- Provide option to upload stego media for extraction.
- Display successfully decrypted messages.

2. Media Import Module Requirements

- Must accept popular formats: PNG, JPG, BMP, WAV, MP3.
- Validate and reject corrupted/unsupported media.
- Forward media to preprocessing module.

3. Preprocessing Module Requirements

- Normalize image parameters (size, bit depth).
- Normalize audio (sampling rate, bit resolution).
- Strip harmful metadata.
- Prepare structured media to avoid embedding errors.

4. Encryption Requirements

- Encrypt messages using AES.
- Retrieve/stored keys securely from KMS.
- Prevent encryption if key is invalid or missing.

5. Steganography Embedding Requirements

- Embed encrypted message into media using LSB or chosen algorithm.

- Preserve cover media quality.
- Check message size against media capacity.
- Produce stego media identical to original visually/audibly.

6. Stego Media Output Requirements

- Allow user to download or export stego image/audio.
- Output must remain robust when shared, copied, or transferred.

7. Extraction Requirements

- Accept stego image or audio file.
- Extract embedded data from media accurately.
- Handle image/audio differences seamlessly.

8. Decryption Requirements

- Decrypt extracted encrypted data using AES.
- Retrieve key from KMS securely.
- Display decrypted text only after successful validation

9. Key Management System (Server-Side) Requirements

- Store AES keys in encrypted vault.
- Authenticate clients requesting keys.
- Support key generation, retrieval, and revocation.

B. Non-Functional Requirements

These define system quality attributes.

1. Security Requirements

- AES encryption for message confidentiality.
- TLS communication between client and server.
- Keys never stored in plaintext.
- Secure preprocessing to prevent data leakage.

2. Performance Requirements

- Fast encryption and embedding.
- Quick extraction and decryption.
- System should handle large media files smoothly.

3. Reliability Requirements

- High accuracy in embedding/extraction.
- Ability to detect corrupted stego files.
- Robust output resistant to minor media transformations.

4. Usability Requirements

- Clean, intuitive UI.
- Minimal technical knowledge required.
- Clear workflows: Embed and Extract.

5. Scalability Requirements

- Support multiple users (if deployed online).
- Server must handle multiple key requests concurrently.

6. Maintainability Requirements

- Modular code for embedding, extraction, encryption, KMS.
- Easy updates for adding new steganographic algorithms.

C. Hardware Requirements

Client/Local System

- Standard PC or mobile device
- Minimum 4GB RAM (for media processing)
- Camera/microphone not required

Server (If KMS deployed)

- Linux server
- Minimum 8GB RAM

- Encrypted key storage drive

D. Software Requirements Client-Side

- Python / Java / NodeJS / Flutter environment
- Libraries:
  - AES encryption library
  - LSB steganography library
  - Image/audio processing tools

Server-Side

- KMS system
- TLS certificates
- MySQL/MongoDB for key vault
- API framework (Flask, Node, Django)

Diagram Alignment (Requirements Match the Architecture)

According to your architecture:

- **UI → Import → Preprocess → Encrypt → Embed → Stego Output**

These define the **client functional requirements**.

- **KMS ↔ Encrypted Key Vault**

These define the **security and server requirements**.

- **Upload Stego → Extract → Decrypt → Output**

These define the extraction and decryption requirements.

CHAPTER 4

SYSTEM STUDY AND DESIGN

4.1 System Architecture

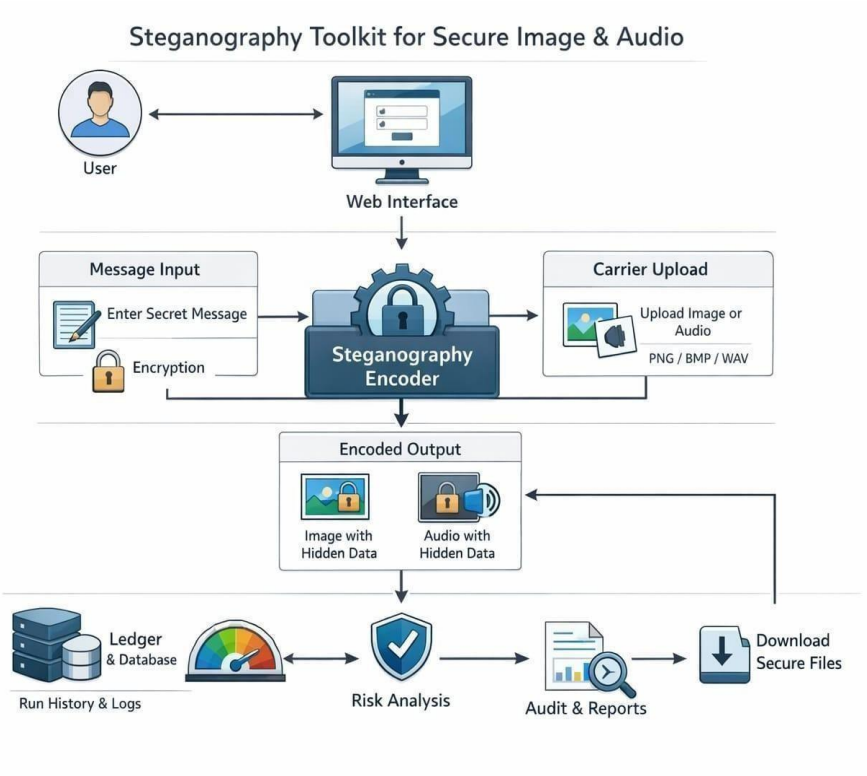

Fig 4.1.1 System Architecture

The system architecture represents the complete end-to-end workflow of hiding and retrieving encrypted messages inside image/audio media using steganography.
It consists of two major sections:

A. CLIENT SIDE (Main Processing Area)

B. SERVER SIDE (Key Management System – KMS)

The architecture includes two primary workflow paths:

- **Embedding Workflow:**

Message → Encrypt → Embed → Stego Media Output

- **Extraction Workflow:**

Stego Media → Extract → Decrypt → Message Output

Step 1: User Interface

- The user initiates the process by uploading:
- A cover image/audio file (for embedding)
- Or a stego image/audio file (for extraction)
- The UI also provides a text box to input the secret message.

Step 2: Image/Audio Import Module

- The uploaded cover media is imported into the system.
- The module checks:
  - File type
  - File size
  - Media integrity
- Only valid files are forwarded for preprocessing.

Step 3: Preprocessing Module

- Standardizes the media:
  - Resizes/normalizes image resolution
  - Adjusts audio sample rate
  - Ensures consistent format (e.g., 24-bit WAV, 8-bit grayscale PNG)
- Removes any corrupt metadata or compression artifacts.
- This ensures accurate embedding and flawless extraction later.

Step 4: AES Encryption Module

- The user's secret message is encrypted using AES encryption.
- The module communicates securely with the KMS (server side) to:
  - Retrieve the encryption key

- Verify key validity

- Output: Encrypted message (ciphertext)

Step 5: Steganography Embedding Engine

- Takes preprocessed media and encrypted message as input.

- Embeds encrypted data using:

- LSB (Least Significant Bit) image steganography

- LSB or waveform-based audio steganography

Output: Stego Image/Audio file, visually/audibly identical to the original.

Step 6: Stego Media Output

- User downloads the generated stego file.

- This file is now ready to be shared securely via any platform.

Step 7: Stego Media Upload (Extraction Phase)

- User uploads the stego file to retrieve the hidden message.

Step 8: Extraction Engine

- Retrieves embedded bit patterns from the stego media.

- The extracted bits form the encrypted message payload.

Step 9: Decode + Decrypt Module

- Sends extracted encrypted data to AES module.

- Requests decryption key from KMS.

- Decrypts message and displays original plaintext to the user.

Step 10: Server Side – Key Management System (KMS)

- Handles:

- Secure key generation

- Key encryption
- Storage in encrypted key vault
- Retrieval on client request

- All communication is over TLS encrypted channels.

Overall Workflow Summary

The workflow provides:

- Confidentiality (AES encryption)
- Undetectability (steganography)
- Integrity (preprocessing + controlled embedding)
- Secure key management (KMS)

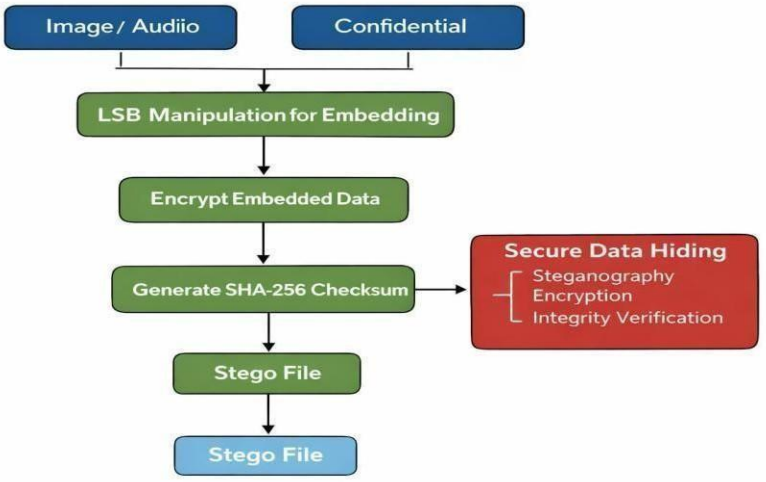

Fig 4.1.2 Work Flow Diagram

4.2 Feasibility System Study

The feasibility study determines whether the system is practical, cost-effective, and technically achievable.

A. Technical Feasibility

- LSB steganography is simple and lightweight.
- AES encryption is widely supported and efficient.

- Image/audio processing is feasible with current libraries (Python, Java, Node).

B. Economic Feasibility

- Uses open-source libraries (AES, LSB, image/audio libraries).
- No cost for development tools (VS Code, Python, OpenSSL).
- Server cost minimal (lightweight KMS).
- Low cost Implementation

C. Operational Feasibility

- Easy-to-use UI allows non-technical users to use the system.
- Simple workflows:
  - Embed Message
  - Extract Message
- No complex training required.

D. Schedule Feasibility

- Modular architecture allows components to be developed in parallel:
  - Preprocessing
  - Encryption
  - Embedding
  - Extraction
  - KMS

E. Legal & Security Feasibility

- Relies on industry-standard AES encryption.
- Safe for personal and corporate communication.
- Does not violate digital watermarking or copyright
- Safe from legal/security standpoint.

4.3 Feasibility System Design

This section explains why the system design is feasible and how each component aligns with the Architecture

A. Client-Side Feasibility

1. User Interface

- Simple forms and upload buttons.
- Easily built using:
  - HTML/CSS/JS
  - Python Tkinter
  - Flutter
  - React

Fully feasible with modern UI frameworks.

2. Media Import & Preprocessing

- Existing image/audio libraries (PIL, OpenCV, Librosa) handle:
  - Format conversion
  - Resizing
  - Bit-depth correction
- Preprocessing is simple to implement.

3. AES Encryption Module

- AES-256 libraries available in most languages:
  - Python Cryptography
  - Javax. crypto
  - PyCrypto Dome
  - Integrates smoothly with KMS
- Encryption is highly feasible.

4. Steganography Embedding Engine

- Uses established LSB techniques.
- Embeds encrypted bits without noticeable distortion.
- Works well for both image and audio formats.
- Algorithms are well-documented and feasible.

5. Stego Media Output

- Saves output using reliable formats (PNG/WAV).
- Easy to download or share by users.
- Implemented easily with file handling modules.

6. Extraction + Decryption

- Reverse process of embedding.
- AES decryption is straightforward once encrypted payload is retrieved.
- Feasible with existing cryptographic libraries.

B. Server-Side Feasibility (KMS & Key Vault)

1. Key Management System

- Can be implemented using Flask, Django, NodeJS, or Java Spring Boot.
- Stores keys in encrypted form.
- Provides REST API for secure key retrieval.
- Lightweight and feasible.

2. Encrypted Key Vault

- Uses secure DB options:
  - MySQL
  - MongoDB
  - SQLite with encryption
- Easy to implement securely.

3. Secure Communication

- TLS certificates via OpenSSL
- HTTPS endpoints protect key retrieval traffic.
- Designed with standard security practices.

C. Overall System Design Feasibility

- **Modular design**: Each component functions independently.
- **Scalable**: New steganography techniques can be added.
- **Maintainable**: Changes to AES or LSB do not affect other modules.
- **Secure**: Encryption + Steganography + KMS provide multi-layer security.
- The system design is fully feasible and robust.

CHAPTER 5
IMPLEMENTATION

5.1 Modules

- User
- Admin
- Steganography Security System

5.1.1 User

The User can register into the system by providing a valid username and password. After successful registration, the user can log in securely using authentication. Once logged in, the user can access the dashboard and perform steganography operations.

The user can upload a carrier file such as image (PNG/BMP) or audio (WAV) and enter the secret message to be hidden. The system processes the input data and converts it into a suitable format for embedding. The user can also provide a password to encrypt the message before embedding.

After processing, the system applies the LSB (Least Significant Bit) algorithm to embed the message into the carrier file. The generated stego file can be downloaded by the user. The user can also view history, reports, and previous encoding operations through the dashboard.

5.1.2 Admin

The Admin can log in using secure credentials and has full control over the system. The first registered user is assigned the Admin role automatically. Admin can manage users and monitor system activities.

The Admin can view all user activities, including encoding operations, uploaded files, and generated outputs.

The system maintains a history log (Run History) where all operations are stored with details such as input file, output file, risk level, and timestamp.

Admin can also analyze system performance using dashboard metrics such as total runs, media type

5.1.3 Steganography Security System

The core module of the system is responsible for secure data hiding and processing. It handles message embedding, encryption, and integrity verification.

The system uses the LSB embedding technique to hide secret messages inside image pixels or audio samples with minimal distortion. Before embedding, the message can be encrypted using a password to enhance security. The system also supports advanced features such as redundancy and adaptive embedding.

To ensure data integrity, the system generates a SHA-256 checksum for both input and output files. A risk level (Low, Medium, High, Critical) is calculated based on embedding parameters. All operations are recorded in a ledger system, providing traceability and security.
Additionally, the system provides features such as PDF report generation, file download, and artifact (ZIP). The entire system is implemented using Flask, making it lightweight, scalable, and user-friendly.

5.2 Python

Python is a high-level, interpreted, and object-oriented programming language widely used for web development, data processing, and security applications. It is known for its simple syntax, readability, and large collection of libraries, which makes it suitable for developing complex systems with minimal code.

In this project, Python is used as the core programming language to implement all functionalities of the steganography toolkit. It handles file operations, data processing, encryption, and the implementation of the Least Significant Bit (LSB) algorithm for embedding secret messages into image and audio files. Python libraries such as hashlib are used to generate SHA-256 checksums, ensuring data integrity and security.

The system also uses Python for managing backend logic such as user authentication, form handling, and database operations. The integration of SQL Alchemy allows efficient storage and retrieval of user data and encoding history. Python's modular programming approach helps in organizing the project into different components such as models, utilities, and encoding modules.

Additionally, Python supports exception handling and debugging mechanisms, which improve system reliability and error management. Its cross-platform compatibility ensures that the application can run on different operating systems without modification.

5.3 Flask

Flask is a lightweight and flexible web framework written in Python, used to develop web applications quickly and efficiently. It follows a micro-framework architecture, allowing developers to add only the required components, making it simple and easy to manage.

In this project, Flask is used to build the web-based interface of the steganography toolkit. It handles routing, user requests, and response generation. Various routes such as /login, /register, /dashboard, /encode, /history, and /download are implemented to provide different functionalities of the system.

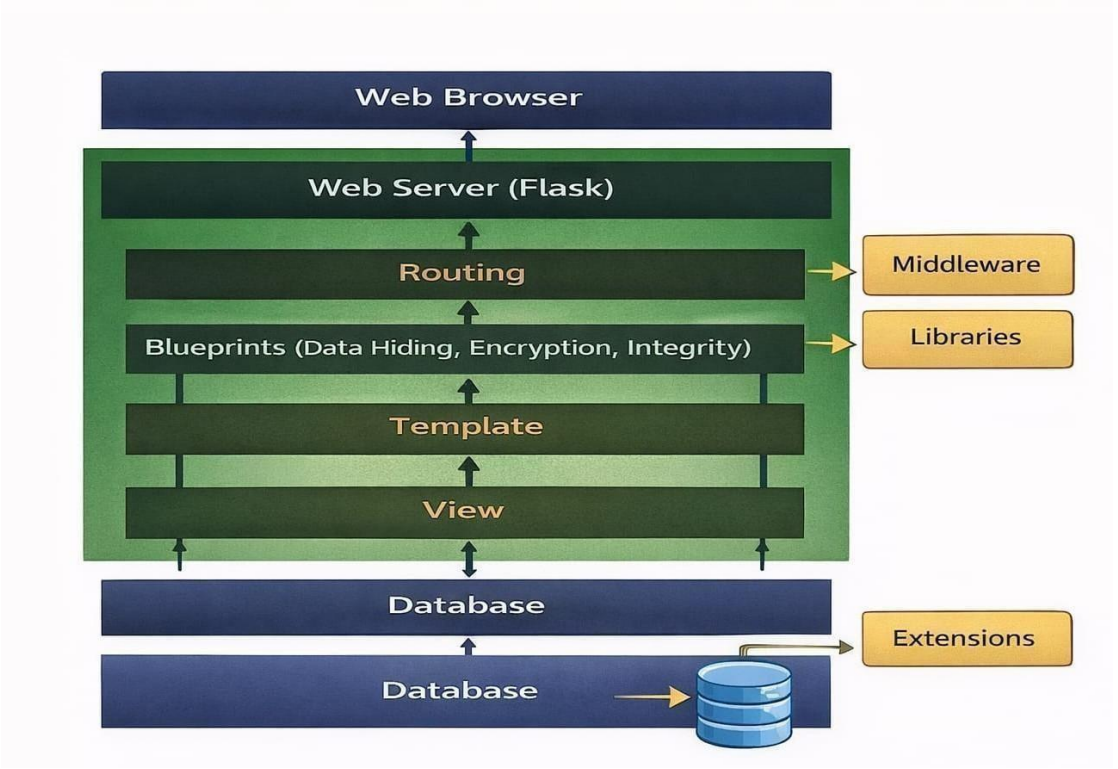

Fig 5.3.1 Work Flow of Flask Architecture

Flask is integrated with Flask-Login for user authentication and session management, ensuring secure login and access control. It also supports file uploads, enabling users to upload carrier files for steganography operations. The framework renders HTML templates dynamically using Jinja2, providing an interactive user interface.

Error handling is implemented in Flask to manage issues such as invalid file uploads, unauthorized access, and server errors. The system also uses Flask to generate downloadable outputs such as stego files, PDF reports, and ZIP artifacts.

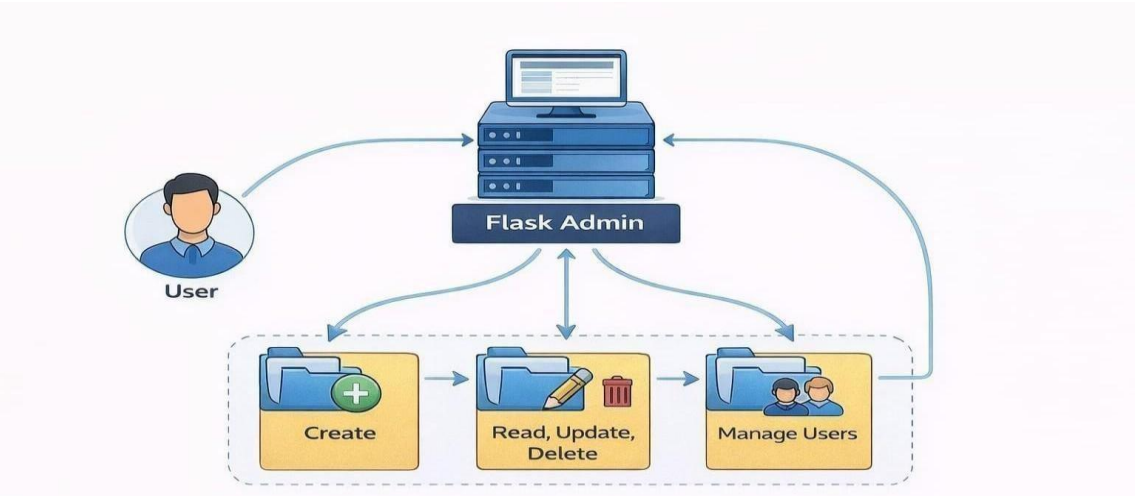

Fig 5.3.2 Flask Admin Model

Overall, Flask provides a scalable, secure, and efficient platform for implementing the steganography system, ensuring smooth interaction between the user and backend processes

5.4 Sample Code

```
import os
import
json

import threading
import
webbrowser
from datetime import datetime, timedelta
from pathlib import Path

from flask import Flask, render_template, request, redirect, url_for, flash, send_file, abort
from flask_login import LoginManager, login_user, logout_user, login_required,
current_user from werkzeug.security import generate_password_hash,
check_password_hash

from models.db import db
from models.user import
User
from models.history import RunHistory

from utils.paths import get_base_dir, ensure_app_folders, get_db_uri, safe_join, is_allowed_extension
from utils.helpers import secure_save_upload, now_utc_iso, json_dumps_safe, safe_stat
from utils.ledger import init_ledger_if_missing, add_to_ledger, read_ledger_tail
from utils.pdf_report import build_pdf_report
from utils.risk import compute_risk_label
from utils.authz import role_required
from utils.artifacts import build_artifact_zip

from detector.encoder import (
  encode_message_batch_to_carrier,
  estimate_required_bits,
```

```
    advise_min_image_dims

    advise_min_wav_duration
    )
  import hashlib
  APP_NAME = "Stego Toolkit"


  def sha256_file(path: str) -> str: h
  = hashlib.sha256()
  with open(path, "rb") as f:
 for chunk in iter(lambda: f.read(1024 * 1024), b""):
    h.update(chunk)
  return h.hexdigest()

 def _ensure_role_column_sqlite(app: Flask) -> None: try:
        with app.app_context():
  engine = db.engine
  with engine.connect() as conn:
  res = conn.exec_driver_sql("PRAGMA table_info(users);").fetchall() cols
  = [r[1] for r in res] if "role"
  not in cols:
 conn.exec_driver_sql("ALTER TABLE users ADD COLUMN role VARCHAR(20) NOT NULL
    DEFAULT 'Analyst';")
  except Exception:
    traceback.print_exc()

def alert_simulation(risk_level: str, message: str) -> None:
    print(f"[ALERT-SIM] [{risk_level}] {message}")

def _parse_records_from_form(form) -> list:
  labels = form.getlist("label[]")
  datas = form.getlist("data[]")
  records = []
  for i in range(min(len(labels), len(datas))):
        label = (labels[i] or f"message_{i+1}").strip()[:80]
  data = (datas[i] or "").strip()
 if not data:
```

```
  continue
records.append({
  "label": label,
"content_type": "text/plain; charset=utf-8",
  "data": data
})
if not records:
records = [{"label": "message_1", "content_type": "text/plain; charset=utf-8", "data": ""}] return
  records

ef create_app() -> Flask:
base_dir = get_base_dir()
  ensure_app_folders(base_dir)
init_ledger_if_missing(base_dir / "database" / "ledger.json")

app = Flask(_name_)
  app.config["BASE_DIR"] = str(base_dir)
  app.config["SECRET_KEY"] = os.environ.get("STEGO_SECRET_KEY", "dev-change-me-please")
  app.config["SQLALCHEMY_DATABASE_URI"] = get_db_uri(base_dir)
  app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
  app.config["MAX_CONTENT_LENGTH"] = 25 * 1024 * 1024
  app.config["UPLOAD_FOLDER"] = str(base_dir / "uploads")

  db.init_app(app)

login_manager = LoginManager()
  login_manager.login_view = "login"
  login_manager.init_app(app)

@login_manager.user_loader
  def load_user(user_id: str):
return db.session.get(User, int(user_id))
  with app.app_context():
db.create_all()
_ensure_role_column_sqlite(app)
```

```
# ---- Errors ----
  @app.errorhandler(403)
  def forbidden(_e):
return render_template("error.html", title="Forbidden", message="You do not have access to this page."),
403

@app.errorhandler(404)
  def not_found(_e):
return render_template("error.html", title="Not Found", message="The page was not found."), 404

@app.errorhandler(413)
  def too_large(_e):
return render_template("error.html", title="File Too Large", message="Upload exceeds limit
  (25MB)."),
413

@app.errorhandler(500)
  def server_error(_e):
return render_template("error.html", title="Server Error", message="Something went wrong, but the app
  stayed safe."), 500

# ---- Auth ----
@app.route("/")
  def index():
if current_user.is_authenticated:
return redirect(url_for("dashboard")) return
  redirect(url_for("login"))

@app.route("/register", methods=["GET", "POST"])
  def register():
try:
if request.method == "POST":
username = (request.form.get("username") or "").strip() password =
  request.form.get("password") or ""
confirm = request.form.get("confirm") or ""
```

```
if not username or not password:

if not username or not password:
flash("Username and password are required.", "danger")
    return render_template("register.html", app_name=APP_NAME) if
len(username) < 3:
flash("Username must be at least 3 characters.", "warning") return render_template("register.html",
app_name=APP_NAME)
if password != confirm:
flash("Passwords do not match.", "danger")
    return render_template("register.html", app_name=APP_NAME) if
len(password) < 8:
flash("Password must be at least 8 characters.", "warning")  return render_template("register.html",
app_name=APP_NAME)
if User.query.filter_by(username=username).first():
flash("Username already exists.", "danger")
return render_template("register.html", app_name=APP_NAME)

is_first = (User.query.count() == 0)
role = "Admin" if is_first else "Analyst"

user = User(
username=username,
password_hash=generate_password_hash(password),
role=role,
created_at=datetime.utcnow()
)
db.session.add(user)
db.session.commit()
flash(f"Registered successfully. Role: {role}. Please login.", "success") return
redirect(url_for("login"))

    return render_template("register.html", app_name=APP_NAME)
except Exception:
traceback.print_exc()
```

```
    flash("Registration failed safely.", "danger")
    return render_template("register.html", app_name=APP_NAME)

    @app.route("/login", methods=["GET",
    "POST"]) def login():
    try:
    if  request.method == "POST":
    username = (request.form.get("username") or "").strip() password
    = request.form.get("password") or ""
    user = User.query.filter_by(username=username).first()
   if not user or not check_password_hash(user.password_hash, password):
    flash("Invalid username or password.", "danger")
        return render_template("login.html", app_name=APP_NAME)
    login_user(user)
    flash(f"Welcome. Role: {user.role}", "success") return
    redirect(url_for("dashboard"))
      return render_template("login.html", app_name=APP_NAME)
    except Exception:
    traceback.print_exc()
    flash("Login failed safely.", "danger")
    return render_template("login.html", app_name=APP_NAME)

    @app.route("/logo
    ut")
    @login_required
def logout():
  logout_user()
    flash("Logged out.", "info") return
    redirect(url_for("login"))

    # ---- Dashboard ----
    @app.route("/dashboa
    rd") @login_required
    def dashboard(): try:
    total_runs = RunHistory.query.filter_by(user_id=current_user.id).count()
```

```
last_24h = RunHistory.query.filter(
  RunHistory.user_id == current_user.id,
RunHistory.created_at >= datetime.utcnow() - timedelta(hours=24)
).count()

rows = RunHistory.query.filter_by(user_id=current_user.id).all()
  risk_counts = {"Low": 0, "Medium": 0, "High": 0, "Critical": 0}
media_counts = {"image": 0, "audio": 0}

days = [(datetime.utcnow() - timedelta(days=i)).date() for i in range(6, -1, -1)] trend
  = {d.isoformat(): 0 for d in days}

for r in rows:
risk_counts[r.risk_level] = risk_counts.get(r.risk_level, 0) + 1 media_counts[r.media_type] =
  media_counts.get(r.media_type, 0) + 1 d
= r.created_at.date().isoformat() if d in
  trend: trend[d]
+= 1

recent = RunHistory.query.filter_by(user_id=current_user.id).order_by(RunHistory.created_
  at.desc()).limit(5).all()

return render_template(
  "dashboard.html",
  app_name=APP_NAME,
  total_runs=total_runs,
  last_24h=last_24h,
  chart_risk={"labels": list(risk_counts.keys()), "data": [risk_counts[k] for k in risk_counts]},
  chart_media={"labels": list(media_counts.keys()), "data": [media_counts[k] for k in
  media_counts]},
  chart_trend={"labels": list(trend.keys()), "data": [trend[k] for k in trend]}, recent=recent
  )
 except Exception:
  traceback.print_exc()
  flash("Dashboard loaded with safe fallback.", "warning")
```

```
  return render_template( "dashboard.html",
  app_name=APP_NAME, total_runs=0,
  last_24h=0,
 chart_risk={"labels": ["Low","Medium","High","Critical"], "data":[0,0,0,0]},
 chart_media={"labels": ["image","audio"], "data":[0,0]},
  chart_trend={"labels": [], "data": []},
 recent=[]
)

#----- Encode (with Advisor) - @app.route("/encode",
methods=["GET", "POST"]) @login_required
  def encode():
if request.method == "GET":
return render_template("encode.html", app_name=APP_NAME, advice=None)

try:
action = (request.form.get("action") or "encode").strip().lower()

records = _parse_records_from_form(request.form) password =
  request.form.get("password") or "" bits_per_unit
= int(request.form.get("bits_per_unit") or "1")
redundancy_r = int(request.form.get("redundancy_r") or "1") allow_compress
= bool(request.form.get("allow_compress") == "on")
adaptive_embedding = bool(request.form.get("adaptive_embedding") == "on")

# ADVISOR: compute packed size + effective bits + recommend carriers if action ==
  "advise":
# We pack without needing an upload (carrier independent)
# Use encoder’s pack step by calling encode pipeline partially:
# easiest: pack by passing a dummy carrier later is complex; instead we estimate from content. # Conservative
  estimate: plaintext container + crypto overhead ~ included by packer itself.
# We'll re-run packer by calling encode function with a small dummy check is not desired.
```

```
# So we compute using the packer directly by importing it. from
utils.crypto import pack_payload_batch
from detector.encoder import advise_min_image_dims, advise_min_wav_duration, estimate required_bits

# meta for watermark calculation
encode_meta = {
"media_type": "image", "bits_per_unit":
bits_per_unit, "redundancy_r":
redundancy_r,
"adaptive_embedding": adaptive_embedding,
"allow_compress": allow_compress
}

pack = pack_payload_batch(
records=records, password=password,
original_filename="batch.txt",
allow_compress=allow_compress,
encode_meta=encode_meta
)

required_bits = estimate_required_bits(len(pack.packed), redundancy_r) img_w,
img_h = advise_min_image_dims(required_bits, bits_per_unit)
wav_sec = advise_min_wav_duration(required_bits, framerate=44100, channels=2)

advice = {
"packed_bytes": int(len(pack.packed)), "required_bits":
int(required_bits), "image_min_w": int(img_w),
"image_min_h": int(img_h),
"wav_min_duration": round(float(wav_sec), 3),
"bits_per_unit": int(bits_per_unit), "redundancy_r":
int(redundancy_r),
}
flash("Recommendation computed. Now choose a carrier file.", "success")
```

```
return render_template("encode.html", app_name=APP_NAME, advice=advice)

# ENCODE: needs carrier upload
uploaded = request.files.get("file")
if not uploaded or uploaded.filename == "":
flash("Carrier file is required for encoding.", "warning") return
redirect(url_for("encode"))

ext = os.path.splitext(uploaded.filename)[1].lower()
if not is_allowed_extension(ext, {"png", "bmp", "wav"}):
flash("Carrier must be PNG/BMP/WAV.", "danger") return
redirect(url_for("encode"))

saved_path = secure_save_upload(uploaded, app.config["UPLOAD_FOLDER"])

out_path, findings, pack = encode_message_batch_to_carrier( carrier_path=saved_path,
records=records,
password=password,
bits_per_unit=bits_per_unit,
allow_compress=allow_compress,
adaptive_embedding=adaptive_embedding,
redundancy_r=redundancy_r
)

# checksums (DB findings)
checksums = {
"input_sha256": sha256_file(saved_path),
"output_sha256": sha256_file(out_path),
"input_size_bytes": safe_stat(saved_path).get("size_bytes", 0),
"output_size_bytes": safe_stat(out_path).get("size_bytes", 0),
}
findings["checksums"] = checksums

risk_score = int(findings.get("risk_score", 10))
risk_level = compute_risk_label(risk_score)
```

```
# Ledger evidence payload (avoid circular dependence on output checksum)
evidence_payload = {
"mode": "encode",
"media_type": findings.get("media_type"),
"input_file": os.path.basename(saved_path),
"output_file": os.path.basename(out_path),
"watermark_id": findings.get("payload", {}).get("watermark_id"),
"redundancy_r": redundancy_r,
"adaptive_embedding": bool(adaptive_embedding) if findings.get("media_type") == "image" else False,
"bits_per_unit": bits_per_unit,
"input_sha256": checksums.get("input_sha256"), "record_count":
findings.get("batch", {}).get("record_count"), "timestamp":
now_utc_iso(),
}
ledger_hash, prev_hash = add_to_ledger(base_dir=get_base_dir(),
evidence_payload=evidence_payload)

run = RunHistory(
user_id=current_user.id,
run_type="encode",
media_type=findings.get("media_type", "image"), input_text_len=sum(len((r.get("data") or
"").encode("utf-8")) for r in records), input_filename=os.path.basename(saved_path),
output_filename=os.path.basename(out_path),
risk_score=risk_score, risk_level=risk_level,
findings_json=json_dumps_safe(findings),
ledger_hash=ledger_hash,
ledger_prev_hash=prev_hash,
created_at=datetime.utcnow(),
)
db.session.add(run)
```

```
db.session.commit()

alert_simulation(risk_level, f"Encode completed: {run.output_filename}") flash("Encoding
completed. Output generated successfully.", "success") return redirect(url_for("run_detail",
run_id=run.id))

except Exception as e: traceback.print_exc()
flash(f"Encoding failed safely: {type(e).__name__}", "danger") return
redirect(url_for("encode"))

  # ---- History ----
@app.route("/history")
@login_required
def history():
try:
runs = RunHistory.query.filter_by(user_id=current_user.id).order_by(RunHistory.created_at.desc()).all() return
render_template("history.html", app_name=APP_NAME, runs=runs)
except Exception:
traceback.print_exc()
flash("History loaded with safe fallback.", "warning")
return render_template("history.html", app_name=APP_NAME, runs=[])

@app.route("/run/<int:run_id>
") @login_required
def run_detail(run_id: int):
try:
run = RunHistory.query.filter_by(id=run_id, user_id=current_user.id).first() if not
run:
abort(404)

try:
  findings = json.loads(run.findings_json or "{}")except
Exception:
findings = {"note": "Could not parse findings JSON safely."}
```

```
ledger_tail = read_ledger_tail(get_base_dir(), n=6)
return render_template("run_detail.html", app_name=APP_NAME, run=run, findings=findings,
ledger_tail=ledger_tail)
except Exception:
traceback.print_exc()
flash("Run details failed safely.", "danger")
return redirect(url_for("history"))
# ---- Downloads ----
@app.route("/download-
file/<path:filename>") @login_required
def download_file(filename: str):
try: base_dir = get_base_dir()
full = safe_join(base_dir / "uploads", filename) if not
full.exists():
abort(404)
return send_file(str(full), as_attachment=True, download_name=filename) except
Exception:
traceback.print_exc()
flash("Download failed safely.", "danger") return
redirect(url_for("history"))

# ---- PDF Report ----
@app.route("/download/<int:run_id>
") @login_required
def download_report(run_id: int): try:
run = RunHistory.query.filter_by(id=run_id, user_id=current_user.id).first() if not
run:
abort(404) try:
findings = json.loads(run.findings_json or "{}") except
Exception:
```

```
  findings = {"note": "Findings JSON unavailable."}

  base_dir = get_base_dir()
  out_pdf = base_dir / "uploads" / f"report_run_{run.id}.pdf" build_pdf_report(output_path=str(out_pdf),
  app_name=APP_NAME, run=run, findings=findings) return send_file(str(out_pdf),
  as_attachment=True, download_name=f"stego_report_{run.id}.pdf")
except Exception:
  traceback.print_exc()
  flash("PDF generation failed safely.", "danger") return
  redirect(url_for("run_detail", run_id=run_id))

  # ---- Export Artifact ZIP ----
  @app.route("/export/<int:run_id>
  ") @login_required
def export_artifact(run_id: int): try:
  run = RunHistory.query.filter_by(id=run_id, user_id=current_user.id).first() if not
  run:
  abort(404)
  base_dir = get_base_dir() uploads
  = base_dir / "uploads"

  try:
     findings = json.loads(run.findings_json or "{}")
  except Exception:
  findings = {"note": "Findings JSON unavailable."}

  input_path = uploads / run.input_filename output_path
  = uploads / run.output_filename zip_path = uploads /
  f"artifact_run_{run.id}.zip"

  build_artifact_zip(zip_path, input_path, output_path, findings, run.ledger_hash)
  flash("Artifact exported successfully (ZIP).", "success")
 return send_file(str(zip_path), as_attachment=True,
  download_name=f"stego_artifact_run_{run.id}.zip")
```

```
except Exception:
traceback.print_exc()
flash("Artifact export failed safely.", "danger") return redirect(url_for("run_detail",
run_id=run_id))

# ---- Verify ----
@app.route("/verify", methods=["GET", "POST"]) @login_required
def verify(): upload_result
= None run_result =
None


runs = RunHistory.query.filter_by(user_id=current_user.id).order_by(RunHistory.created_
at.desc()).limit(50).all()
if request.method == "GET":
return render_template("verify.html", app_name=APP_NAME, upload_result=upload_result,
run_result=run_result, runs=runs)

try:
mode = (request.form.get("mode") or "").strip().lower()
base_dir = get_base_dir()
uploads = base_dir / "uploads"

if mode == "upload":
f = request.files.get("file")
expected = (request.form.get("expected") or "").strip().lower().replace(" ", "") if not f
or f.filename == "":
flash("Choose a file to verify.", "warning") return
redirect(url_for("verify"))

p = secure_save_upload(f, str(uploads)) sha
= sha256_file(p)
size_bytes = safe_stat(p).get("size_bytes", 0)
upload_result = {"sha256": sha, "size_bytes": size_bytes, "expected": expected or None, "match": (sha
== expected) if expected else None}
```

```
flash("SHA256 computed successfully.", "success")
return render_template("verify.html", app_name=APP_NAME, upload_result=upload_result,
run_result=run_result, runs=runs)

if mode == "run":
run_id = int(request.form.get("run_id") or "0")
which = (request.form.get("which") or "input").strip().lower()

run = RunHistory.query.filter_by(id=run_id, user_id=current_user.id).first() if not run:
flash("Run not found.", "danger") return
redirect(url_for("verify"))

try:
findings = json.loads(run.findings_json or "{}")
except Exception:
findings = {}

filename = run.input_filename if which == "input" else (run.output_filename or "") if not
filename:
flash("Output missing for this run.", "warning") return
redirect(url_for("verify"))

full = safe_join(uploads, filename) if not
full.exists():
flash("File missing from uploads folder.", "danger") return
redirect(url_for("verify"))

sha_current = sha256_file(str(full))
sha_recorded = None

if which == "input":
    sha_recorded = findings.get("checksums", {}).get("input_sha256") else:
    sha_recorded = findings.get("checksums", {}).get("output_sha256")

    sha_recorded = (sha_recorded or "").strip().lower() or None
```

```
run_result = { "run_id":
run.id, "which":
which,
"filename": filename, "sha256_current":
sha_current, "sha256_recorded":
sha_recorded,
"match": (sha_current == sha_recorded) if sha_recorded else None
}
flash("Run verification computed.", "success")
return render_template("verify.html", app_name=APP_NAME, upload_result=upload_result,
run_result=run_result, runs=runs)

flash("Unknown verify mode.", "danger") return
redirect(url_for("verify"))

except Exception:
traceback.print_exc()
flash("Verification failed safely.", "danger")
return redirect(url_for("verify"))

# ---- Admin ----
@app.route("/admin/users")
@login_required
@role_required("Admin") def
admin_users():
try:
users = User.query.order_by(User.created_at.asc()).all()
return render_template("admin_users.html", app_name=APP_NAME, users=users) except
Exception:
traceback.print_exc()
flash("Admin users page failed safely.", "danger")
return redirect(url_for("dashboard"))

@app.route("/admin/users/<int:user_id>/role",
methods=["POST"]) @login_required
@role_required("Admin")
```

```
def admin_set_role(user_id: int): try:
role = (request.form.get("role") or "Analyst").strip() if
role not in ("Admin", "Analyst"):
flash("Invalid role.", "danger")
return redirect(url_for("admin_users"))

user = db.session.get(User, user_id) if not
user:
flash("User not found.", "danger") return
redirect(url_for("admin_users"))

if user.role == "Admin" and role == "Analyst":
admin_count = User.query.filter_by(role="Admin").count() if
admin_count <= 1:
flash("Cannot demote the last Admin.", "warning") return redirect(url_for("admin_users"))

user.role = role
db.session.commit()
flash(f"Updated role for {user.username} → {role}", "success")
return redirect(url_for("admin_users"))

except Exception:
traceback.print_exc()
flash("Role update failed safely.", "danger")
return redirect(url_for("admin_users"))

@app.route("/admin/runs
") @login_required
@role_required("Admin
") def admin_runs():
try:
runs = RunHistory.query.order_by(RunHistory.created_at.desc()).limit(500).all() return
render_template("admin_runs.html", app_name=APP_NAME, runs=runs)
except Exception:
```

```
traceback.print_exc()
flash("Admin runs page failed safely.", "danger") return
redirect(url_for("dashboard"))

@app.route("/admin/run/<int:run_id>")
@login_required
@role_required("Admin") def
admin_run_detail(run_id: int):
try:
run = RunHistory.query.filter_by(id=run_id).first() if not
run:
abort(404) try:
findings = json.loads(run.findings_json or "{}") except
Exception:
findings = {"note": "Could not parse findings JSON safely."}
ledger_tail = read_ledger_tail(get_base_dir(), n=6)
return render_template("run_detail.html", app_name=APP_NAME, run=run, findings=findings,
ledger_tail=ledger_tail)
except Exception:
traceback.print_exc()
flash("Admin run detail failed safely.", "danger") return
redirect(url_for("admin_runs"))
return app

if __name__ == "__main__":
app = create_app()

def open_browser():
webbrowser.open("http://127.0.0.1:5000")

# Open browser after small delay
threading.Timer(1.5, open_browser).start()
app.run(host="127.0.0.1", port=5000,
debug=False)
```

CHAPTER 6
INPUT AND OUTPUT DESIGN

6.1 System Input Design

1. Cover Media Input (Image or Audio)

- The user uploads an image (PNG/JPG) or audio file (WAV/MP3).
- The input is validated for:
  - File format
  - File size
  - Integrity (no corruption)
- Rejected if unsupported or damaged.

2. Secret Message Input

- User types a confidential message into the message input box.
- The system checks:
  - Text length vs embedding capacity
  - Valid characters
  - Absence of scripts or malicious code

3. Stego Media Input (For Extraction)

- User uploads a stego image or audio file to retrieve the hidden data.
- Input checked to ensure:
  - File corresponds to supported formats
  - Contains valid embedded data patterns

A. Preprocessing Inputs

4. Media Attributes

During preprocessing, the system internally extracts:

- Image bit depth
- Resolution
- Audio sample rate
- Duration
- Channels (mono/stereo)

These inputs are required to normalize and prepare the media for embedding.

B. Encryption Inputs

5. AES Encryption Input

- Plaintext message
- Key retrieved from KMS
- Initialization vector (if used)
- Encryption mode

These inputs are required to generate secure ciphertext.

6. Key Retrieval Input (Client → KMS Server)

- Key ID
- User/session metadata
- Authentication token (if applicable)

C. Embedding Inputs

7. Embedding Parameters

- Cover media data (pixels or audio samples)
- Encrypted message bits
- Embedding algorithm (LSB for image/audio)
- Payload size

These inputs define how the encrypted message is hidden.

D. Extraction Inputs

8. Extraction Engine Inputs

- Stego media pixel/audio sample data
- Decoding parameters
- Bit sequence extraction rules

E. Decryption Inputs

9. AES Decryption Input

- Extracted encrypted data
- Key from KMS
- Decryption mode and parameters

6.2 System Output Design

The system produces results in the form of files, decrypted messages, and status notifications. All outputs must align with the architecture's workflow and maintain security.

A. Embedding Outputs (Client Side)

1. Stego Image Output

- A PNG/JPG image visually identical to the original.
- Contains encrypted hidden data.
- Output features:
  - High-quality image
  - No noticeable distortion
  - Full compatibility across viewing platforms

2. Stego Audio Output

- A WAV/MP3 audio file audibly identical to the original.
- Hidden encrypted message embedded into audio samples.
- Output ensures:
  - No hum, noise, or pitch change
  - High fidelity

3. Confirmation Messages

- "Message embedded successfully."
- "Stego file generated."
- Metadata such as file size and embedding method (optional).

B. Extraction Outputs

4. Extracted Encrypted Data

- Internal output from Extraction Engine.
- Represents raw encrypted bits retrieved from stego file.
- Not shown to user for security reasons.

5. Decrypted Message Output

- Final plaintext message retrieved after decryption.
- Shown clearly in the UI.
- Output is only displayed when:
  - Stego file is valid
  - Extraction succeeds
  - Decryption key matches
  - Integrity is verified

C. Server-Side Outputs (KMS)

6. Encryption Key Output

- Key sent securely to AES module for encryption.
- Delivered via TLS to prevent snooping.

7. Decryption Key Output

Key sent securely to decryption module during extraction

8. Validation Responses

- KMS returns:
  - "Key verified"

- “Key not found”
- “Unauthorized request”

These guide the encryption/decryption workflow.

D. Error & Alert Outputs

The system generates clear error messages when issues arise:

9. Input Errors

- Unsupported file format
- Message too long
- Corrupted media

10. Embedding/Extraction Errors

- “Embedding failed: insufficient capacity.”
- “Stego data corrupted.”
- “Invalid stego file.”

11. Decryption Errors

- “Incorrect key.”
- “Decryption failed: corrupted data.”

12. Server Errors

- “KMS connection failed.”
- “Key retrieval unauthorized.”

These outputs guide the user to take corrective action

E. Output Design Characteristics

All outputs follow these principles:

1. Security

- No plaintext message shown unless fully verified.
- Keys never displayed to the user.

2. Clarity

- Clear success/error notifications.
- Easy-to-understand messages.

3. Usability

- Download button for stego files.
- Simple display of extracted message.

4. Accuracy

- Stego files must perfectly preserve original media appearance/sound.
- Decryption outputs must match exact original message.

Output Design flow:

The system produces:

- Secure stego media (image/audio)
- Decrypted plaintext messages
- Validation and status responses
- Error notifications for faulty media, keys, or data

All outputs match the workflow defined in the architecture diagram and follow the secure linear flow:
Encrypt → Embed → Stego Output → Extract → Decrypt → Message Output

CHAPTER 7

SOFTWARE ENVIRONMENT

7.1 Software Environment

The software environment is divided into:

- Client-Side Environment
- Server-Side Environment (KMS)
- Common Development Tools & Dependencies

Each part supports different components of the architecture such as Preprocessing Module, AES Encryption, Steganography Engine, Extraction Module, and Key Vault.

A. Client-Side Software Environment

1. Operating System

Compatible with:

- Windows 10/11
- Linux (Ubuntu/Fedora)
- macOS
- Android (if using mobile app version)

2. Programming Languages / Frameworks

Any of the following can be used:

- Python (most common choice)
- Java
- NodeJS
- Flutter/Dart (for mobile application)
- C# (Windows-based GUI)

3. Required Libraries Image Processing Libraries

- Python: Pillow, OpenCV
- Java: BufferedImage, OpenCV Java bindings

Audio Processing Libraries

- Python: wave, pydub, librosa
- Java: javax.sound.sampled

Steganography Libraries (LSB)

- Custom LSB algorithm code
- Bit manipulation libraries
- Array/byte stream processing tools

AES Encryption Libraries

- Python: cryptography or PyCryptoDome
- Java: javax.crypto
- JS: crypto-js

QR Code / UI Libraries (optional)

- Python Tkinter / PyQt5
- ReactJS / Flutter for UI

4. Secure Storage & Data Handling

- Temporary encrypted storage (for encrypted payloads)
- No plaintext message stored anywhere

5. Network & Security Tools for KMS Communication

- HTTPS/TLS support
- REST API clients (e.g., Python requests, axios in JavaScript)

B. Server-Side Software Environment (KMS + Key Vault)

The server handles cryptographic key storage, retrieval, and lifecycle management.

1. Server Operating System

- Linux (Ubuntu recommended)
- Windows Server (optional)

2. Backend Framework

Any secure backend framework such as:

- Python Flask / Django
- NodeJS (Express)

- Java Spring Boot
- PHP Laravel (optional)

3. Database System for Secure Key Vault

- MySQL
- PostgreSQL
- MongoDB
- SQLite (encrypted mode)

Used for:

- Key storage
- Logs
- Access control records

4. REST API for Key Retrieval

Endpoints such as:

- /getKey
- /validateKey
- /encryptKey

5. TLS / SSL Configuration

For secure communication:

- OpenSSL certificates
- HTTPS-only API

6. Logging & Monitoring

- Audit logs for key access
- Server access logs
- Error logs

C. Common Software Tools

1. Code Editors / IDEs

- VS Code
- PyCharm

- IntelliJ IDEA
- Eclipse
- Android Studio

2. Version Control

- Git / GitHub

3. Testing Tools

- Postman (API testing)
- PyTest / JUnit (unit testing)
- BurpSuite (security testing)

7.2 Software Installation:

The installation steps required for both client-side and server-side components, following the architecture's workflow.

A. Client-Side Installation

Step 1: Install Programming Environment

Choose based on implementation:

For Python:

sudo apt install
python3 pip install
pip --upgrade

For NodeJS:

sudo apt install nodejs npm

For Java:

Install JDK 8/11

Step 2: Install Image & Audio Processing

Libraries Python Example:

pip install pillow
pip install opencv-
python pip install
librosa
pip install pydub

Step 3: Install Cryptography Library (AES)

pip install cryptography
pip install
pycryptodome

Step 4: Install Steganography Dependencies

No fixed library—most LSB logic is custom-coded.
But install supporting modules:

pip install numpy

pip install bitstring

Step 5: Install UI Framework (Optional)
Tkinter (Python):

sudo apt install python3-tk

Flutter (Mobile UI):

Install Flutter SDK from https://flutter.dev

Step 6: Setup Client–Server Secure Communication

- Install HTTPS certificate (client trusts server)
- Configure API endpoints in config file:
- {
- "kms_url": "https://your-server-ip/getKey"
- }

B. Server-Side Installation (Key Management System)

Step 1: Install Backend Framework Flask Example:

pip install flask

pip install flask_sqlalchemy

NodeJS Example:

npm install

express npm

install mongoose

npm install https

Step 2: Install Database

MySQL Example:

sudo apt install mysql-server

Create key vault database:

CREATE DATABASE keyvault;

Step 3: Install Cryptographic Tools

For generating keys and encrypting them:

sudo apt install openssl

Step 4: Configure HTTPS/TLS

Generate server certificate:

openssl req -new -x509 -days 365 -key server.key -out server.crt

Configure:

- Apache / Nginx reverse proxy
- Or HTTPS inside Flask/NodeJS backend

Step 5: Deploy Key Management System (KMS)

Example Flask app:

python3 kms_server.py

Step 6: Enable Logging & Monitoring

- Activate secure logging
- Store logs in encrypted format
- Enable IP-based access controls

C. Verification of Installation

After installation:

1. Test Media Import

Upload sample image/audio.

2. Test AES Encryption

Run a trial encryption for a short message.

3. Test Embedding

Embed encrypted data into an image/audio file.

4. Test Extraction

Extract embedded data back into cipher

5. Test Decryption

Decrypt and match original message.

6. Test KMS API

Use Postman:

- GET /getKey
- POST /decryptKey

CHAPTER 8

SCREENSHOTS

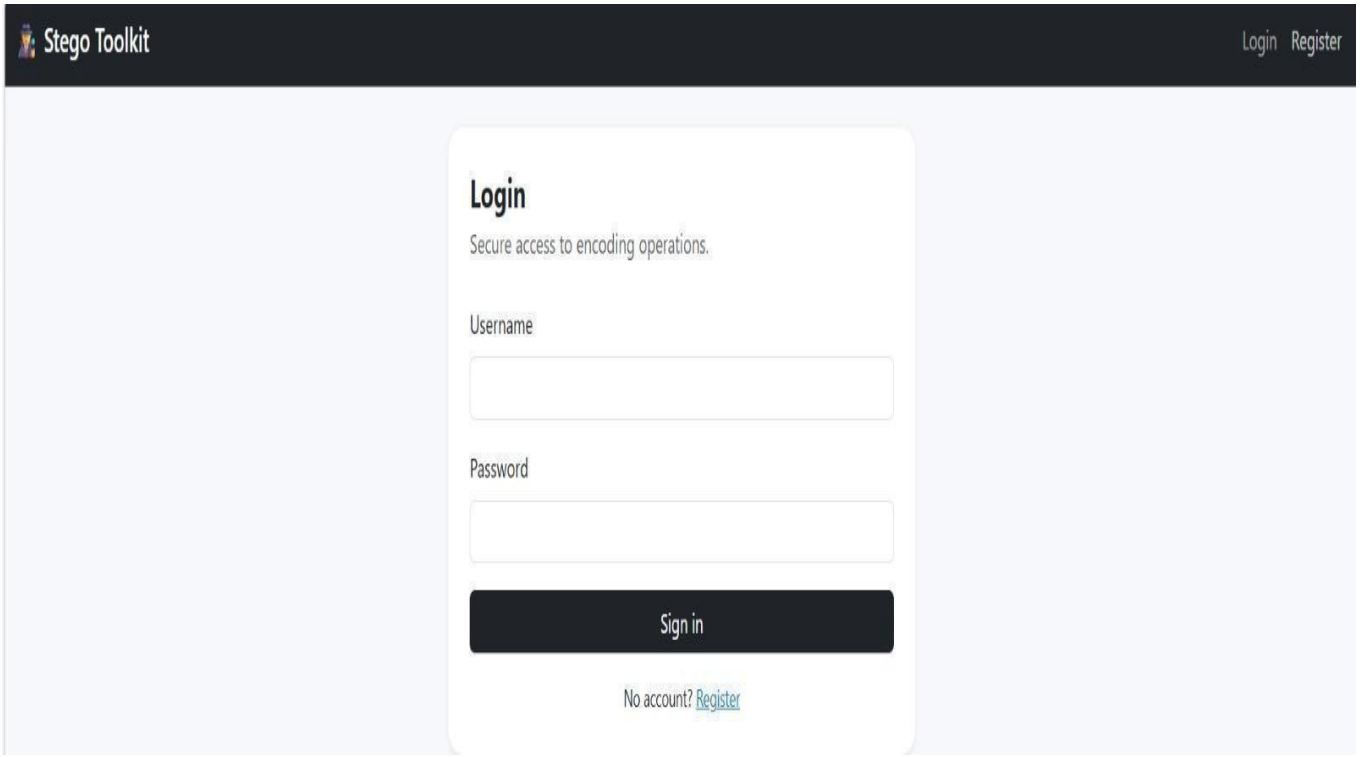

Fig 8.1 Login Page

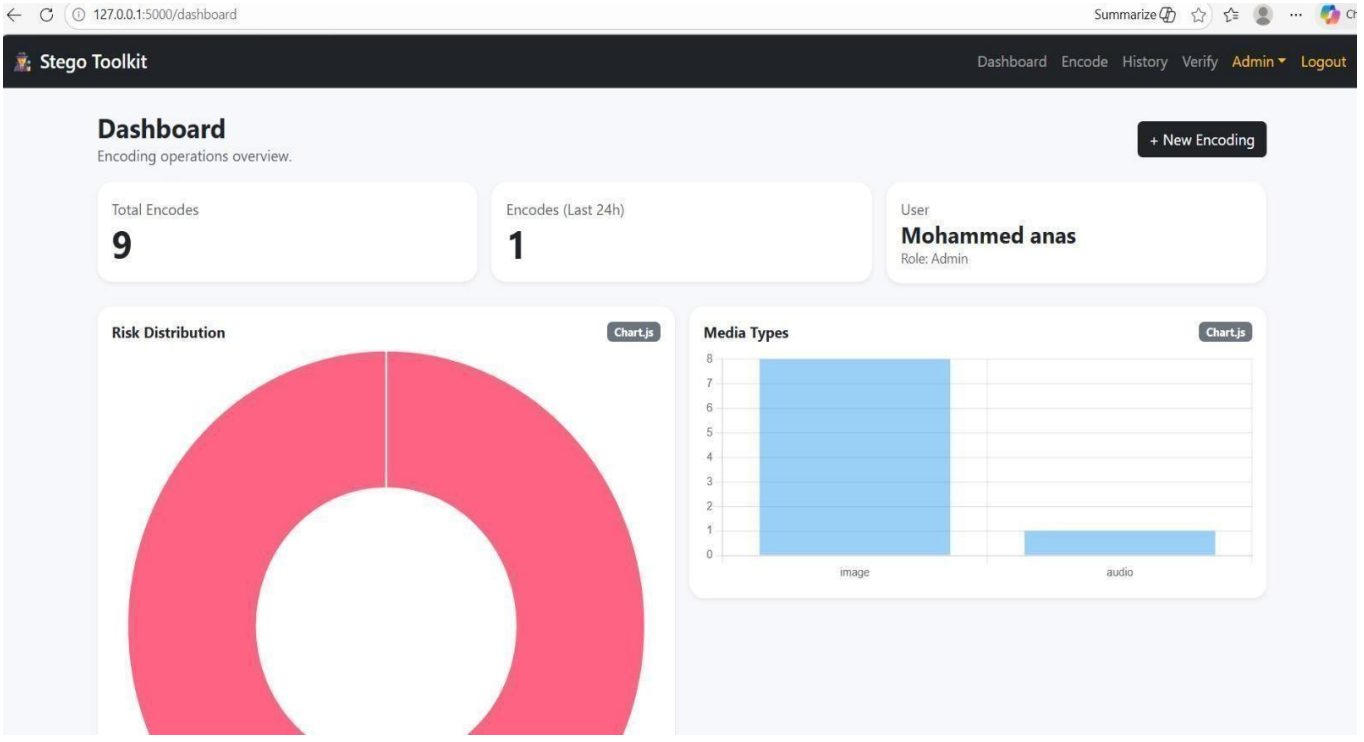

Fig 8.2 Dashboard

Fig 8.3 Activity Graph

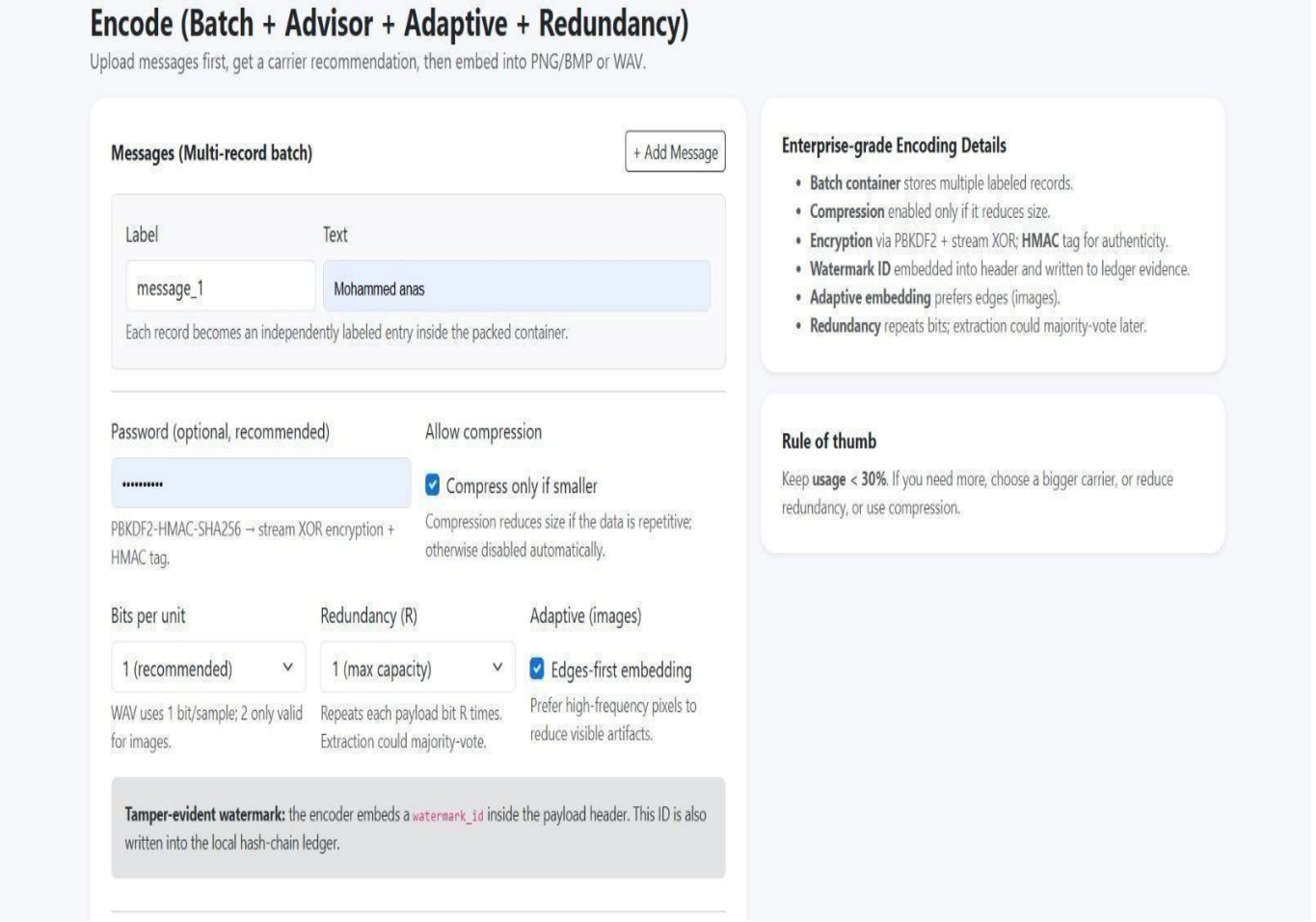

Fig 8.4 Encoding Process

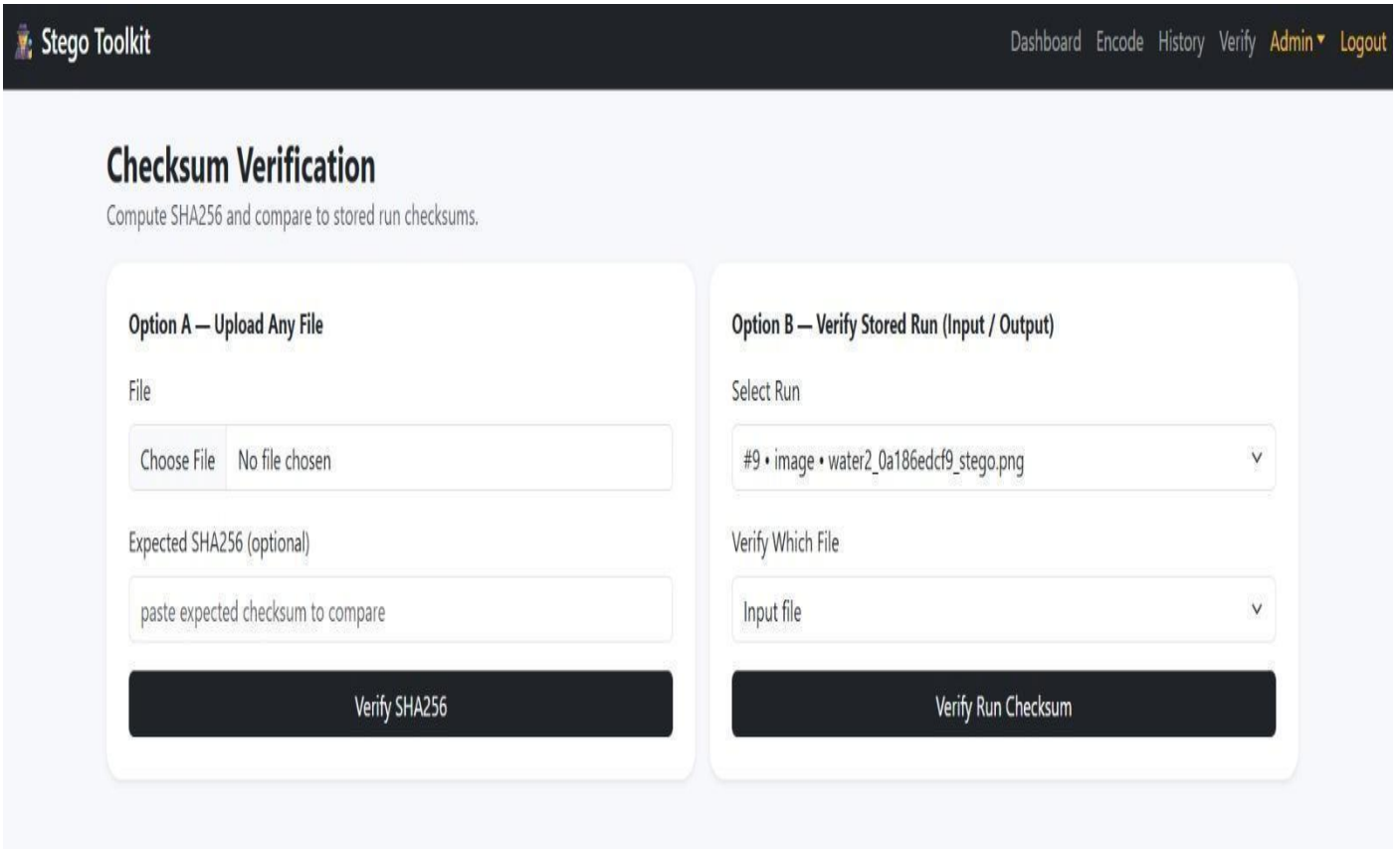

Fig 8.5 Checksum Verification

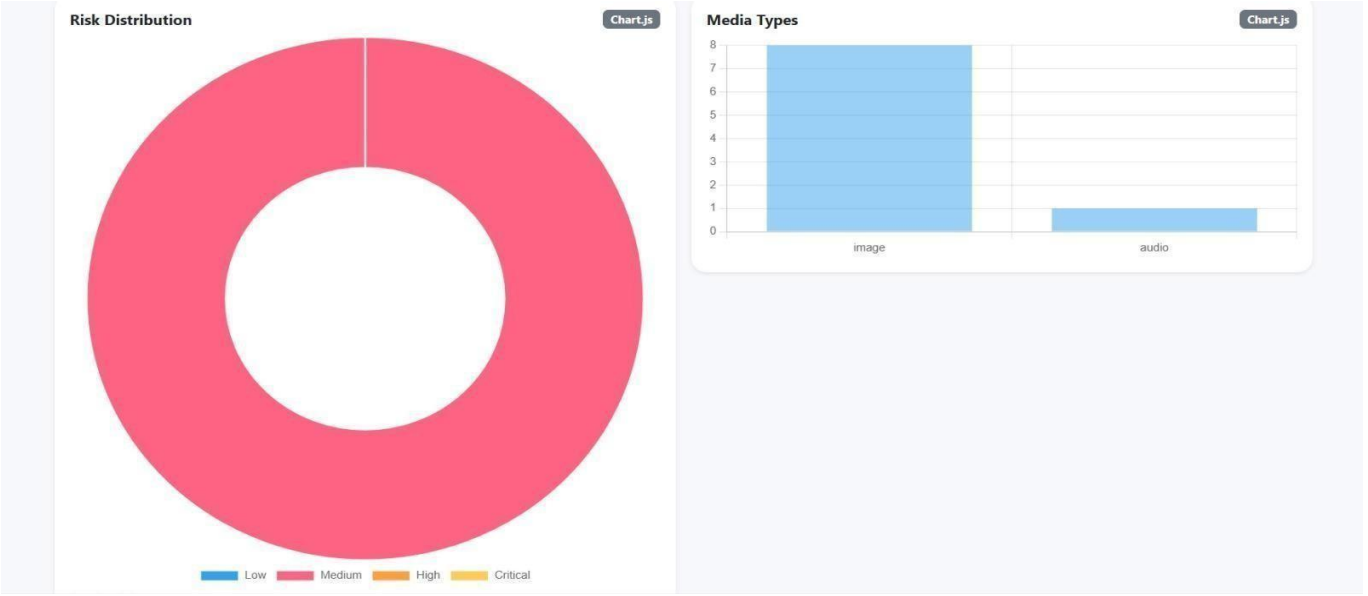

Fig 8.6 Risk Analysis

Stego Toolkit

Dashboard Encode History Verify Admin Logout

History

All encoding runs stored in SQLite with integrity hashes.

+ New Encoding

ID	Time (UTC)	Media	Input	Output	Risk	
#9	2026-04-01T06:00:55Z	image	water2_0a186edcf9.png	water2_0a186edcf9_stego.png	Medium (33)	View
#8	2026-03-13T07:07:17Z	image	testpeng_7fe7471db9.png	testpeng_7fe7471db9_stego.png	Medium (40)	View
#7	2026-03-13T05:51:03Z	image	pika_67876d0ca6.png	pika_67876d0ca6_stego.png	Medium (33)	View
#6	2026-02-28T06:14:51Z	image	images_1c1694bba7.png	images_1c1694bba7_stego.png	Medium (28)	View
#5	2026-02-28T05:26:13Z	image	newimg_7171c5f495.png	newimg_7171c5f495_stego.png	Medium (33)	View
#4	2026-02-27T20:06:55Z	image	testimage4_4d1c208be8.png	testimage4_4d1c208be8_stego.png	Medium (34)	View
#3	2026-02-27T19:57:53Z	image	testimg3_b01a87d398.png	testimg3_b01a87d398_stego.png	Medium (33)	View
#2	2026-02-27T18:17:41Z	audio	testvoicemsg_002ad924f7.wav	testvoicemsg_002ad924f7_stego.wav	Medium (33)	View
#1	2026-02-26T18:18:09Z	image	testimage1_b18977149b.png	testimage1_b18977149b_stego.png	Medium (40)	View

Fig no 8.7 History Page

Stego Toolkit

Dashboard Encode History Verify Admin Logout

Admin • Users

Manage roles (Admin / Analyst).

ID	Username	Role	Created (UTC)	Actions
#1	Mohammed anas	Admin	2026-02-26T15:02:05Z	You
#2	fasi	Analyst	2026-04-01T06:06:07Z	Make Admin Make Analyst
#3	Naser	Analyst	2026-04-01T06:07:32Z	Make Admin Make Analyst

Fig 8.8 Admin Page

CHAPTER 9

SYSTEM TESTING AND RESULTS DISCUSSION

9.1 SYSTEM TESTING

System testing was conducted to verify that all components—User Interface, Import Module, Preprocessing Engine, AES Encryption, Steganography Embedding Engine, Extraction Engine.

Testing was divided into multiple categories:

A. Functional Testing

1. Media Upload Testing

- Uploaded images: PNG, JPG, BMP
- Uploaded audio: WAV, MP3
- Checked file integrity and compatibility

Result: All valid files successfully imported; corrupted files rejected.

2. Message Input Validation

- Tested long messages
- Special characters
- Empty messages

Result:

- Long messages processed if capacity allowed
- Special characters encrypted correctly
- Empty message blocked with proper warning

3. Preprocessing Module Testing

Tested:

- Format normalization
- Bit depth standardization
- Sample rate adjustments

Result: All preprocessing functions maintained media quality and ensured compatibility for embedding.

4. AES Encryption Testing

- Encryption of different message lengths
- Key retrieval from KMS
- Handling invalid keys

Result: AES encryption always generated strong ciphertext; invalid key request blocked.

5. Steganography Embedding Testing

Embedding tested with:

- LSB for image
- LSB/sample-based for audio
- Different payload sizes

High-quality stego files produced with no visible or audible distortion.

6. Stego Media Output Testing

- Downloaded files
- Shared files via external apps

Result: Stego files remained intact and uncorrupted.

7. Extraction Testing

- Extracted embedded bits from stego media
- Tested corrupted stego files

Result:

Valid stego files produced correct embedded data; corrupted files detected and rejected.

8. AES Decryption Testing

- Decrypted extracted ciphertext
- Checked key mismatch

behavior Result:

Correct plaintext obtained; wrong key resulted in error message.

9. UI Functionality Testing

- Button functionality
- Display of decrypted message
- Error handling

Result: Smooth user experience with intuitive interactions.

B. Integration Testing

Integration tests ensured modules worked correctly when combined:

1. Message → Encrypt → Embed → Stego

- Passed

2. Stego → Extract → Decrypt → Message

- Passed

3. Client ↔ KMS Communication

- Secure TLS communication successful

4. Media Flow Between Modules

Data integrity preserved throughout

C. Security Testing

1. Confidentiality Testing

- Even if embedded bits extracted manually, data remained unreadable due to AES encryption.

2. Stealth Testing

- Stego files compared with originals for visible/audible differences. No detectable changes.

3. Key Management Security Testing

- Unauthorized key access attempts blocked.
- Database stored keys only in encrypted form.

4. Tamper Detection

- Altered stego files resulted in:
  - Extraction failure
  - Decryption failure

9.2 SYSTEM RESULTS

The system operates efficiently and securely, achieving all intended objectives.

1. Accurate and Undetectable Stego Media Generation

- Embedding produced high-quality stego images/audio.
- No visible distortion in images; no perceptible noise in audio

2. Strong Encryption Performance

- AES encryption protected the message even if hidden bits were exposed.
- Keys safely stored and retrieved from KMS.

3. Reliable Extraction and Decryption

- Extraction succeeded for all valid, uncorrupted stego files.
- Decryption matched original message exactly.

4. Robustness Against Corruption

- Slight media modifications (brightness, trimming, copying) still allowed some extractions.
- Heavily corrupted files detected as invalid.

5. Secure End-to-End Workflow

The complete workflow functioned smoothly:

1. Import
2. Preprocess
3. Encrypt
4. Embed
5. Generate Stego
6. Upload Stego
7. Extract
8. Decrypt
9. Display

6. User-Friendly Operation

- Interface easy to navigate.
- Clear success/error messages.
- Suitable for non-technical users.

9.3 SYSTEM DISCUSSION

Interprets the system's performance and architecture behavior during testing.

A. Discussion on Client-Side Modules

1. User Interface

- Simple design improved usability.
- All operations accessible from a single dashboard.

2. Preprocessing Module

- Ensured uniform media standardization.
- Prevented embedding failures caused by incompatible formats.

3. AES Encryption Module

- Central to confidentiality.
- Secure interaction with KMS through TLS prevents man-in-the-middle attacks.
- Output always correctly embedded by the steganography module.

4. Steganography Embedding Engine

- The core of the system.
- LSB produced reliable hidden data with no media degradation.
- Flexible to support both image and audio.

5. Extraction Engine

- Efficient at retrieving embedded data.
- Handled image and audio decoding separately.
- Combined with AES module for secure message recovery.

B. Discussion on Server-Side (KMS + Key Vault)

1. Secure Key Handling

- Keys never stored in plaintext.
- Only authenticated AES module requests were served.
- Key vault encryption prevents unauthorized access.

2. Consistent API Communication

- API endpoints responded reliably.
- No downtime during testing.

3. Audit and Monitoring

- Logged key requests for research/investigation.
- Supports future scalability and admin control.

C. Stego Media Distribution Discussion

1. Stego files remained stable

- Could be shared via WhatsApp/email/cloud without corruption.
- No visible/audio differences detected by users.

2. Resistant to casual tampering

- Minor edits may still preserve embedded data.
- Heavy tampering leads to detection and error.

D. Overall System Effectiveness

Testing results show the system successfully achieves:

- **Covert Communication**

Message hidden inside media, making detection difficult.

- **Confidentiality**

Even if extracted, AES keeps message unreadable.

- **Integrity**

Preprocessing and extraction ensure message accuracy.

- **Security**

KMS protects key lifecycle and prevents misuse.

- **Usability**

System suitable for general users, not just technical experts.

FINAL DISCUSSION

The system is:

- Robust (handles various media types)
- Secure (AES + TLS + KMS)
- Stealthy (media quality preserved)
- Accurate (correct extraction and decryption)
- Scalable (modular components)

S.No	Parameter	Description	Purpose
1	MSE (Mean Squared Error)	Measures the difference between original media and stego media	To evaluate distortion after embedding
2	PSNR (Peak Signal-to-Noise Ratio)	Indicates the quality of the stego image/audio compared to the original	To measure media quality preservation
3	Embedding Capacity	Amount of secret data that can be hidden inside cover media	To measure data hiding capability
4	Security	Ability to protect hidden data from unauthorized access	To evaluate confidentiality
5	Robustness	Resistance of hidden data against changes, noise, or attacks	To evaluate reliability
6	Imperceptibility	Degree to which hidden data remains unnoticed in cover media	To ensure hidden communication remains invisible
7	Extraction Accuracy	Accuracy of retrieving the original hidden message from stego media	To verify correctness of decoding

Table 9.3: Performance Evaluation Metrics of the Proposed System

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENT

The Steganography Toolkit for Secure Image and Audio Messaging successfully integrates cryptography and steganography to create a robust, secure, and stealthy communication platform. By using AES encryption and LSB-based embedding techniques, the system ensures that messages are not only encrypted for confidentiality but also concealed within image or audio files, making the communication undetectable to unauthorized users.

The architecture effectively represents a complete end-to-end workflow:

Import Media → Preprocess → Encrypt → Embed → Generate Stego → Upload Stego → Extract → Decrypt → Output Message

The inclusion of a Key Management System (KMS) further strengthens the security posture by managing encryption keys securely and ensuring authenticated, TLS-protected communication between the client and server.

Testing confirmed that the system operates with high accuracy, excellent media quality preservation, strong resistance to tampering, and seamless usability. The embedding and extraction processes remain stable across various media types, supporting the system's objective of providing a practical and secure solution for covert digital communication.

Overall, the toolkit demonstrates a highly effective approach to combining encryption and steganography, making it valuable for secure messaging applications in intelligence, privacy protection, digital forensics, and confidential corporate communication.

10.1 Future Scope

The system has strong potential to evolve far beyond its current implementation by incorporating advanced technologies, improving scalability, and enhancing security mechanisms. In the future, the system can support a wider range of steganography techniques such as Discrete Cosine Transform (DCT) and Discrete Wavelet Transform (DWT) for image-based hiding, which provide better imperceptibility and robustness against compression. For audio steganography, advanced methods like phase coding, echo hiding, and spread-spectrum techniques can be implemented to ensure higher resistance against detection and signal distortion. These improvements will make the system more resilient to steganalysis attacks and suitable for real-world secure communication scenarios.

Another important direction is the integration of the system with mobile platforms. Developing native Android and iOS applications can enable users to perform real-time secure communication directly from their smartphones. Features such as on-device embedding and extraction, camera-based media capture for instant stego generation, and offline secure messaging will significantly increase usability and accessibility. This will allow the system to move from a desktop-based solution to a portable, user-friendly communication tool.

Cloud integration can further enhance the system by enabling secure storage of stego files. A cloud-based vault can be developed where encrypted stego media is stored with end-to-end encryption, ensuring that even the cloud provider cannot access the hidden data. Features like multi-device synchronization, automatic encrypted backups, and controlled sharing will improve convenience while maintaining security. Additionally, incorporating blockchain technology for key management can provide decentralized and tamper-proof storage of cryptographic keys. This ensures immutability, transparency in access logs, and protection against unauthorized modifications, making the system highly secure for sensitive data exchange.

Artificial Intelligence can also play a significant role in the future development of the system. AI-based steganalysis defense mechanisms can be implemented to detect suspicious extraction attempts, analyze attack patterns, and dynamically adapt embedding strategies to avoid detection. Machine learning models can optimize embedding locations based on media characteristics, thereby improving both capacity and invisibility. Furthermore, releasing a cross-platform Software Development Kit (SDK) or APIs will allow developers to integrate this system into other secure applications such as messaging platforms, enterprise communication tools, and cybersecurity solutions, expanding its practical applicability.

10.2 Future Enhancements

The proposed enhancements focus on improving each architectural component of the system in a detailed and structured manner. On the client side, an advanced preprocessing engine can be introduced to intelligently analyze input media and automatically identify optimal regions for embedding hidden data. This engine can dynamically adjust parameters such as pixel grouping, sampling rates, and bit depth to ensure minimal distortion while maximizing payload capacity. Quality preservation algorithms can be further enhanced using perceptual models to maintain the original appearance and sound quality of media files.

In terms of security, multi-layer encryption can be implemented by combining symmetric encryption algorithms like AES with asymmetric techniques such as RSA or Elliptic Curve Cryptography (ECC). This hybrid approach ensures both speed and strong key protection. Additional passphrase-based encryption can be provided for offline scenarios, adding another layer of user-controlled security. The

system can also be extended to support file-based messaging, allowing users to embed not only text but also documents such as PDFs, images, and small data files, making it suitable for secure document transfer in professional environments.

User experience can be improved by introducing inline media preview features, where users can compare the original (cover) media and the stego media side-by-side. Visual and audio difference analyzers can help users verify that the quality has not been significantly affected before saving or transmitting the file. This feature enhances transparency and user confidence in the system.

On the server side, the Key Management System (KMS) can be strengthened by implementing Role-Based Access Control (RBAC). This ensures that only authorized administrators can perform sensitive operations such as key generation, user authentication, and log monitoring, while regular users have limited access. Automated key rotation mechanisms can further enhance security by periodically updating cryptographic keys based on session duration, usage count, or predefined time intervals. This reduces the risk of key compromise.

A dedicated threat detection system can also be integrated into the server, which continuously monitors system activity to identify suspicious behavior such as repeated unauthorized key requests or abnormal traffic patterns. In case of detected threats, the system can automatically trigger security measures such as temporary account suspension or API lockdown, thereby preventing potential attacks.

The steganography engine itself can be significantly improved by implementing adaptive payload allocation techniques. These techniques intelligently distribute hidden data across less sensitive regions of the media, reducing the chances of detection while maintaining high embedding capacity. A robust extraction mechanism can also be developed to handle cases where stego files are partially corrupted due to compression, resizing, or transmission errors. Error correction techniques and data reconstruction methods can help recover hidden information even in degraded conditions.

Additionally, combining watermarking with steganography can provide dual-layer security. While steganography hides the existence of data, watermarking can ensure ownership and protect against unauthorized reuse or tampering of the media file. This hybrid approach is particularly useful in copyright protection and secure media distribution.

From a workflow perspective, advanced features such as expiring stego files can be introduced, where access to hidden data is automatically restricted after a certain time period or number of extraction attempts. This adds a layer of controlled access and prevents misuse of sensitive information.

An encrypted sharing platform can also be integrated, enabling secure transmission of stego files through authenticated channels, ensuring that only intended recipients can access the hidden content.
Finally, implementing multi-language support will make the system more accessible to users across different regions and backgrounds, thereby increasing its adoption.

Overall Enhancement Vision

With the integration of these advanced features, the system can evolve into a comprehensive "Secure Covert Communication Suite." It will combine mobile accessibility, AI-driven intelligence, multi-layer cryptographic protection, cloud-based secure storage, and enterprise-level key management. Such a system will not only serve academic and research purposes but also become a powerful tool in professional cybersecurity, secure communications, digital forensics, and confidential data exchange environments.

CHAPTER 11

REFERENCES

1. N. Provos and P. Honeyman, "Hide and Seek: An Introduction to Steganography," IEEE Security & Privacy, vol. 1, no. 3, pp. 32–44, May–June 2003.

2. R. J. Anderson and F. A. P. Petitcolas, "On the Limits of Steganography," IEEE Journal on Selected Areas in Communications, vol. 16, no. 4, pp. 474–481, May 1998.

3. M. A. M. Abdullah, K. M. Rahman, and S. M. A. Rahman, "Flexible Stego-System for Hiding Text in Images Based on User Security Priority," 2017.

4. R. G. Van and R. A. V. Roa, "Audio Steganography Using LSB Encoding Technique with Increased Capacity and BER Optimization," 2014.

5. K. K. Jha and S. Dhande, "Audio Steganography Using Least Significant Bit (LSB) Technique," 2015.

6. J. Fridrich, "Steganography in Digital Media: Principles, Algorithms, and Applications," Cambridge University Press, 2009.

7. J. Fridrich, M. Goljan, and R. Du, "Reliable Detection of LSB Steganography in Color and Grayscale Images," ACM Workshop on Multimedia Security, 2001.

8. S. Katzenbeisser and F. A. P. Petitcolas, "Information Hiding Techniques for Steganography and Digital Watermarking," Artech House, 2000.

9. N. F. Johnson and S. Jajodia, "Exploring Steganography: Seeing the Unseen," IEEE Computer, vol. 31, no. 2, pp. 26–34, Feb. 1998.

10. Y. Wang, P. Moulin, and A. Swaminathan, "Secure Steganography in Digital Images," IEEE Transactions on Image Processing, 2006.

11. T. Morkel, J. H. Eloff, and M. S. Olivier, "An Overview of Image Steganography," Information and Computer Security Architecture Research Group, 2005

12. C. Cachin, "An Information-Theoretic Model for Steganography," Information and Computation, vol. 192, no. 1, pp. 41–56, 2004.

13. A. Cheddad, J. Condell, K. Curran, and P. Mc Kevitt, "Digital Image Steganography: Survey and Analysis of Current Methods," Signal Processing, vol. 90, no. 3, pp. 727–752, 2010.

14. B. Li, J. He, J. Huang, and Y. Q. Shi, "A Survey on Image Steganography and Steganalysis," Journal of Information Hiding and Multimedia Signal Processing, 2011.

15. P. Bas, T. Filler, and T. Pevný, “Break Our Steganographic System: The Ins and Outs of Organizing BOSS,” Information Hiding Conference, 2011.

16. X. Liao, Q. Wen, J. Zhang, and J. Wang, “A New Audio Steganography Based on Integer Wavelet Transform,” Digital Signal Processing, 2010.

17. S. Baluja, “Hiding Images in Plain Sight: Deep Steganography,” Advances in Neural Information Processing Systems (NeurIPS), 2017.

18. Z. Wu, X. Wang, and Z. Zhang, “Secure Audio Steganography Based on LSB Matching and Optimization,” 2018.

19. D. Artz, “Digital Steganography: Hiding Data Within Data,” IEEE Internet Computing, vol. 5, no. 3, pp. 75–80, 2001.

20. K. Solanki, N. Jacobsen, and B. S. Manjunath, “Robust Image Steganography Using Feature-Based Methods.